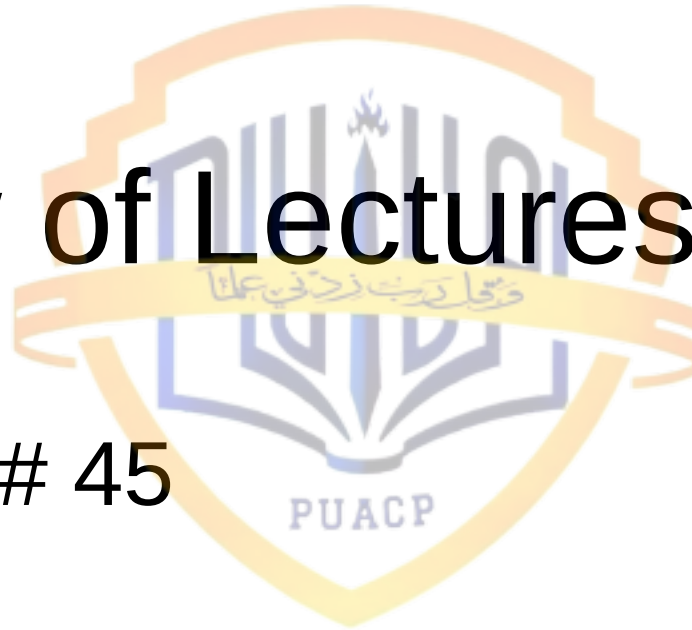
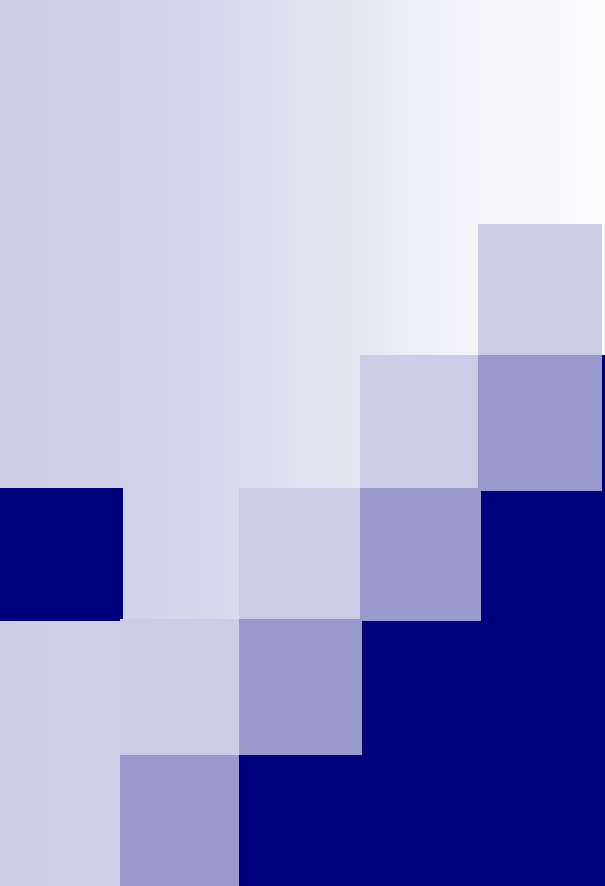


Review of Lectures 23-44

Lecture # 45





Software Testing

Lectures 23-32

Basics of Software Testing - 2

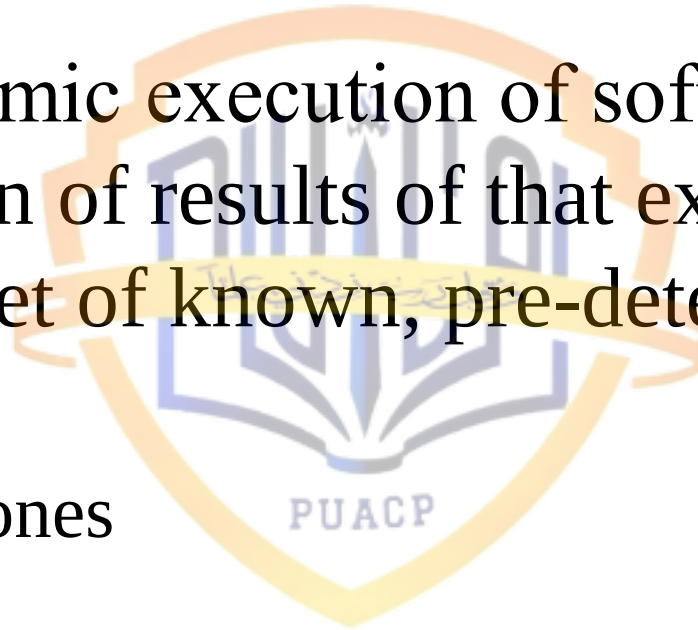
- Testing is one of the most important parts of quality assurance and the most performed QA activity
- Testing has been a basic form of defect removal since software industry began
- For majority of software projects, it is the *only* form of defect removal utilized

Basics of Software Testing - 3

- The basic idea of testing involves the execution of software and the observation of its behavior or outcome
- If a failure is observed, the execution record is analyzed to locate and fix the faults that caused the failure
- Otherwise, we gain some confidence that the software under testing is more likely to fulfill its designated functions

Software Testing

- “The dynamic execution of software and the comparison of results of that execution against a set of known, pre-determined criteria”
 - Capers Jones



Primary Purposes of Software Testing

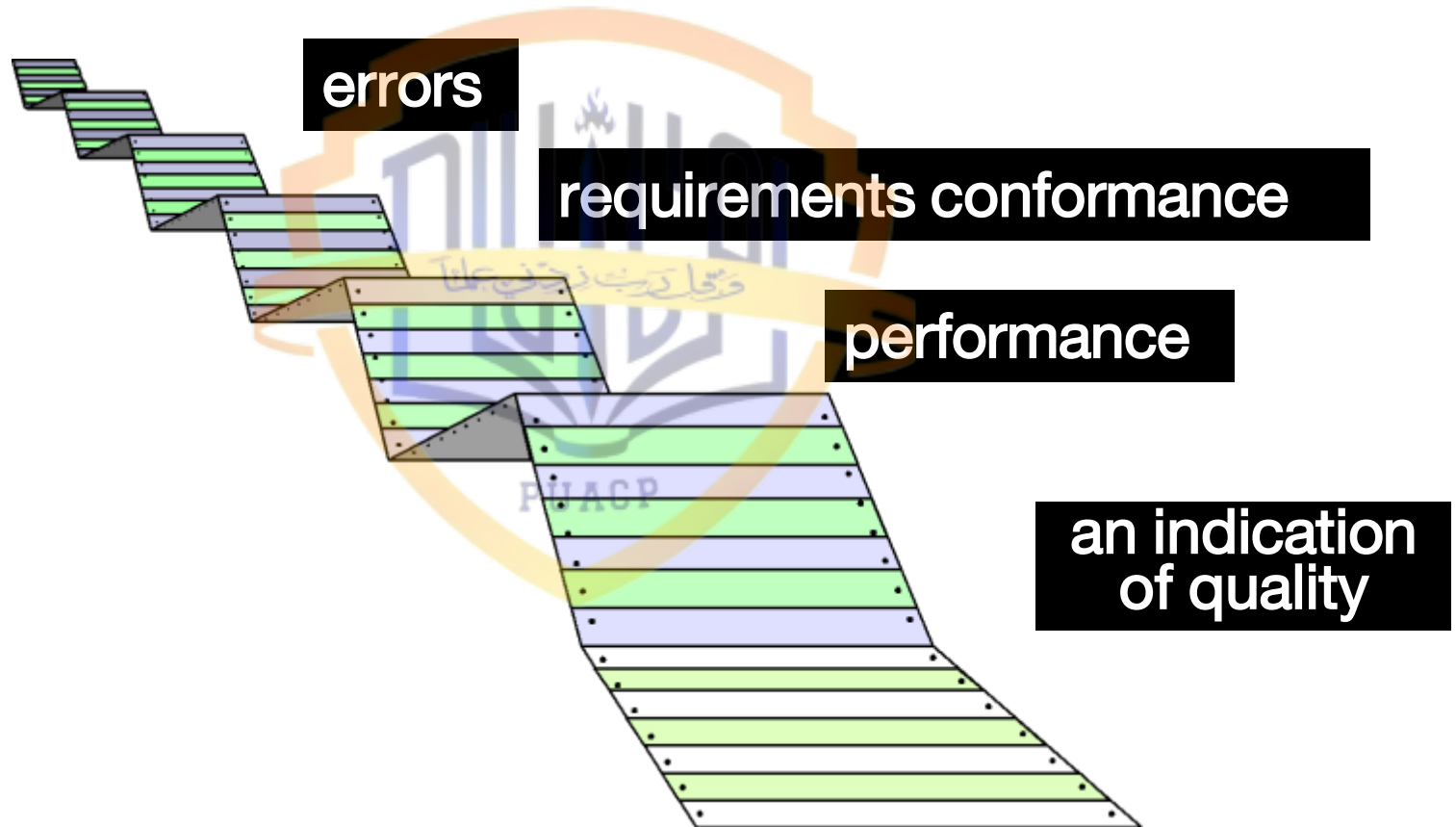
- To demonstrate quality or proper behavior
- To detect and fix problems





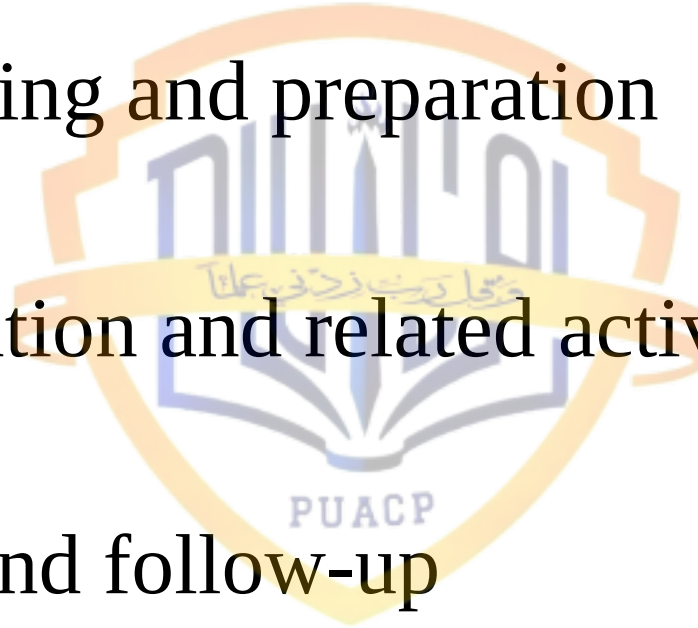
What Testing Shows?

What Testing Shows?



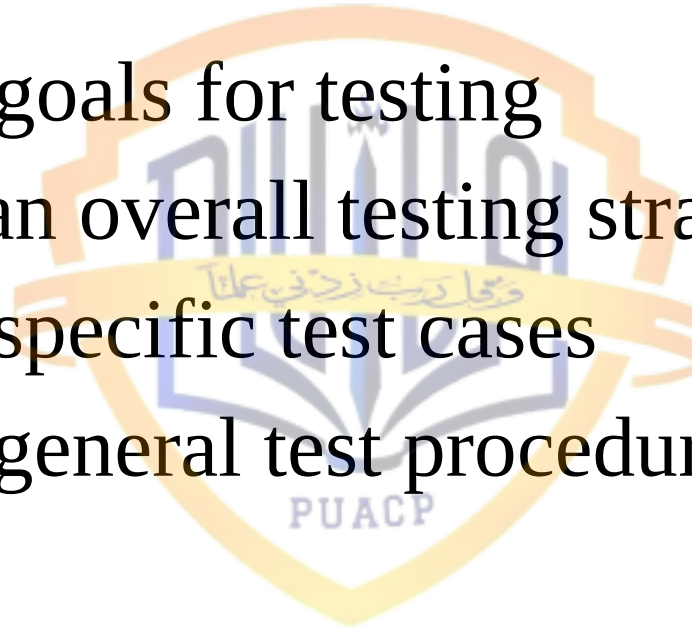
Major Activities and Generic Testing Process

- Test planning and preparation
- Test execution and related activities
- Analysis and follow-up



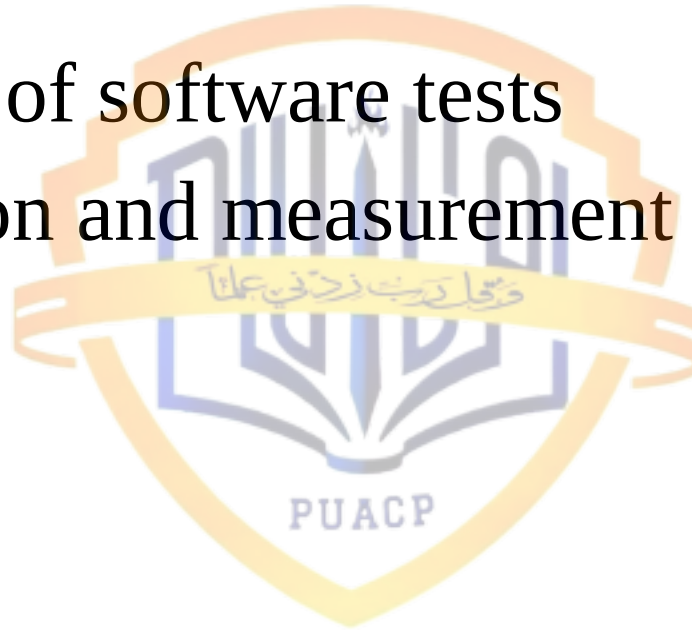
Test Planning and Preparation

- Setting of goals for testing
- Selecting an overall testing strategy
- Preparing specific test cases
- Preparing general test procedure



Test Execution

- Execution of software tests
- Observation and measurement of product behavior



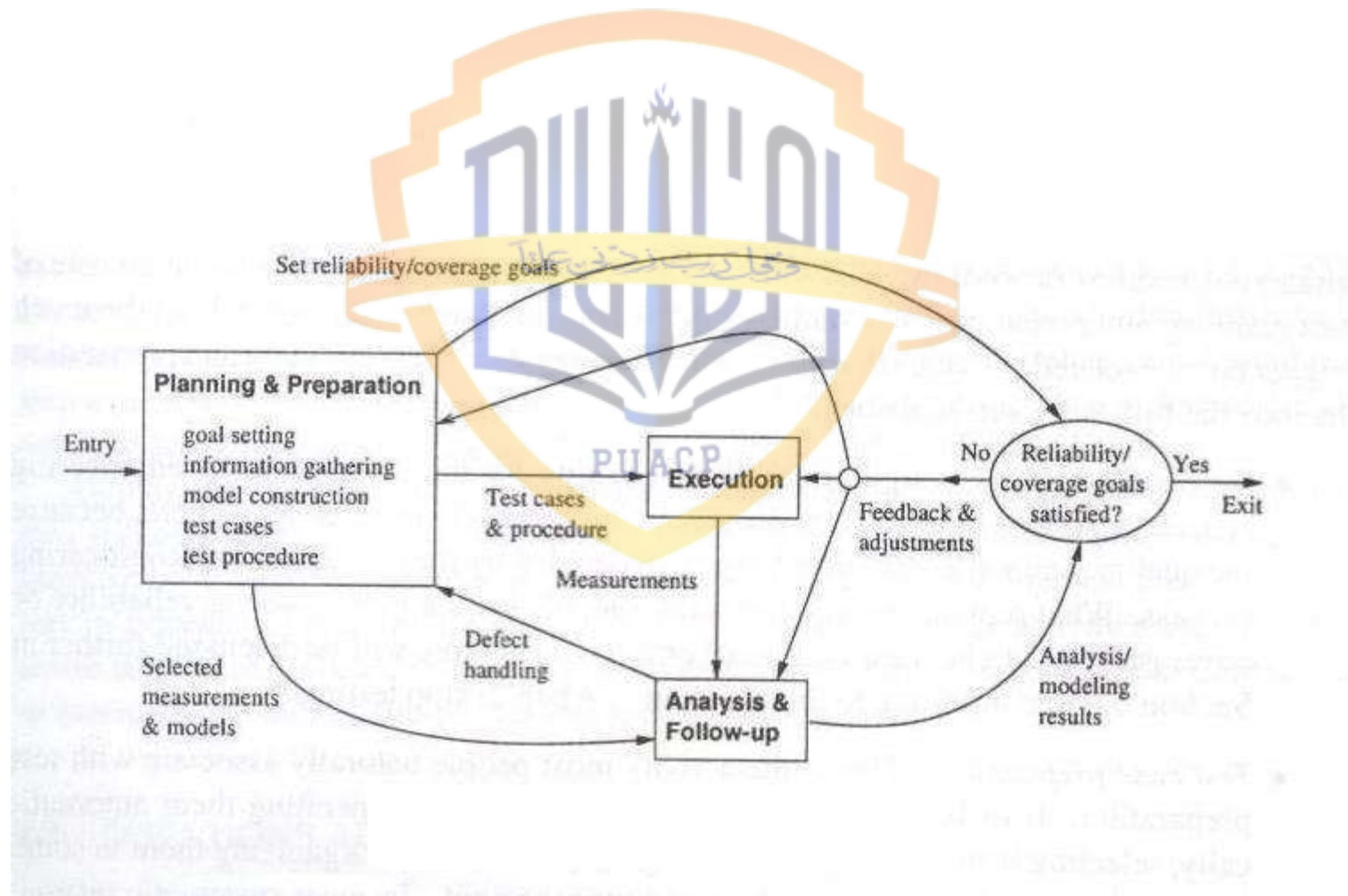
Analysis and Follow-Up

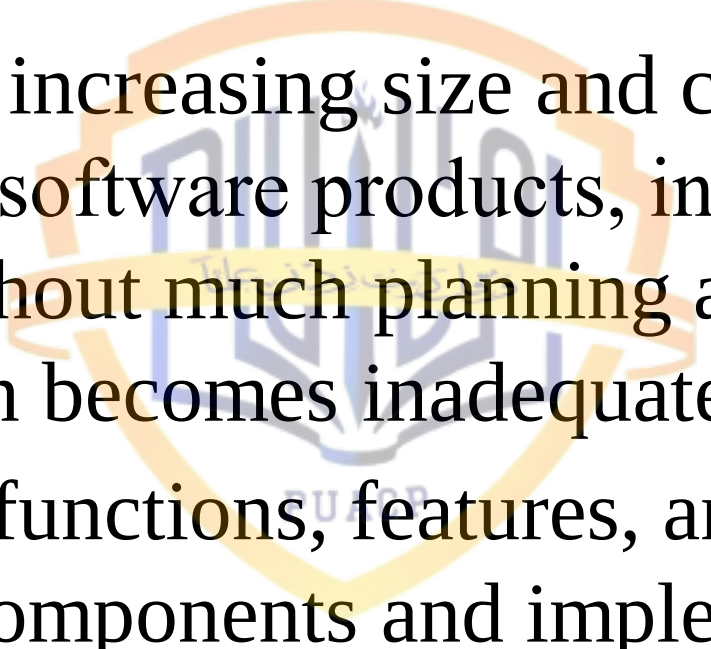
- Includes result checking and analysis to determine if a failure has been observed, and if so, follow-up activities are initiated and monitored to ensure removal of the underlying causes, of faults, that led to the observed failures in the first place

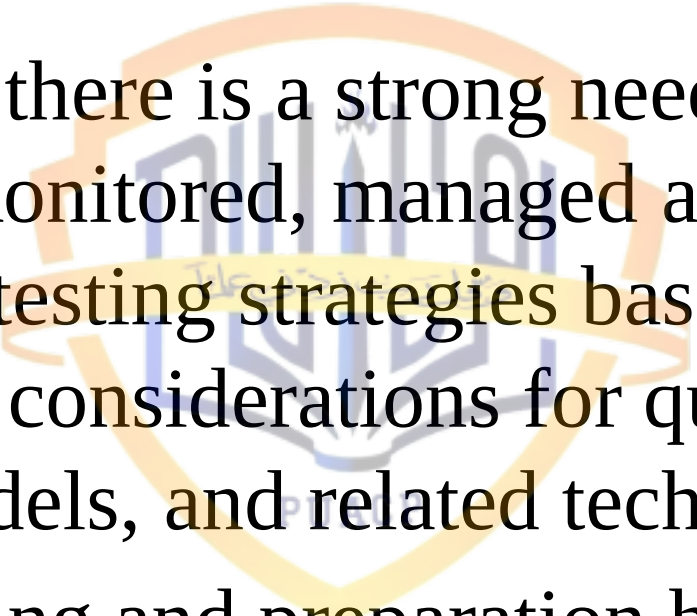


Generic Testing Process

Generic Testing Process



- 
- Due to the increasing size and complexity of today's software products, informal testing without much planning and preparation becomes inadequate
 - Important functions, features, and related software components and implementation details could be easily overlook in such informal testing

- 
- Therefore, there is a strong need for planned, monitored, managed and optimized testing strategies based on systematic considerations for quality, formal models, and related techniques
 - Test planning and preparation has these sub-activities

Sub-Activities in Test Planning and Preparation

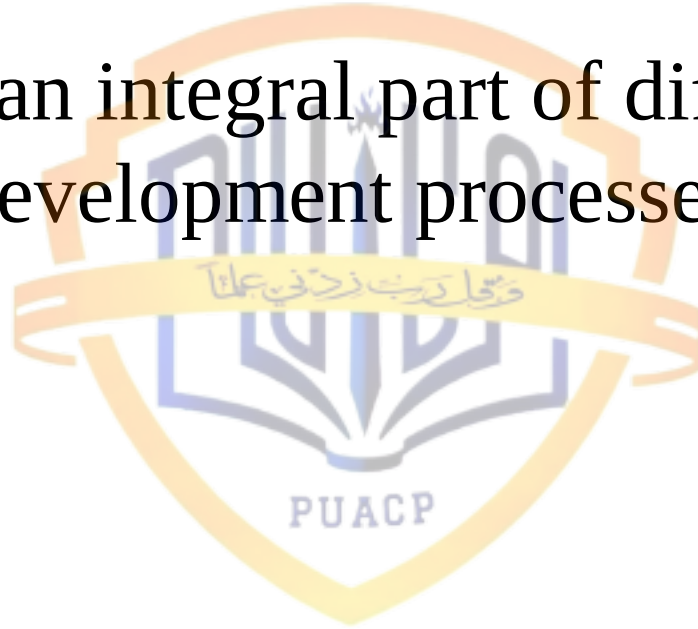
- Goal setting
 - Reliability and coverage goals
- Test case preparation
 - Constructing new test cases or generating automatically, selecting them from existing ones for legacy products, and organizing them in some systematic manner
- Test procedure preparation
 - It is defined and followed to ensure effective test execution, problem handling and resolution, and overall test process management

Testing as a Part of QA in Overall Software Process - 1

- Testing and inspections often find different kinds of problems, and may be more effective under different circumstances
- Therefore, inspections and testing should be viewed more as complementary QA alternatives instead of competing ones

Testing as a Part of QA in Overall Software Process - 2

- Testing is an integral part of different software development processes



Testing and Defect Removal Efficiency

- Most forms of testing are less than 30% efficient in finding bugs
- Many forms of testing should be used in combination with pre-test design and code inspections to achieve high defect removal efficiencies

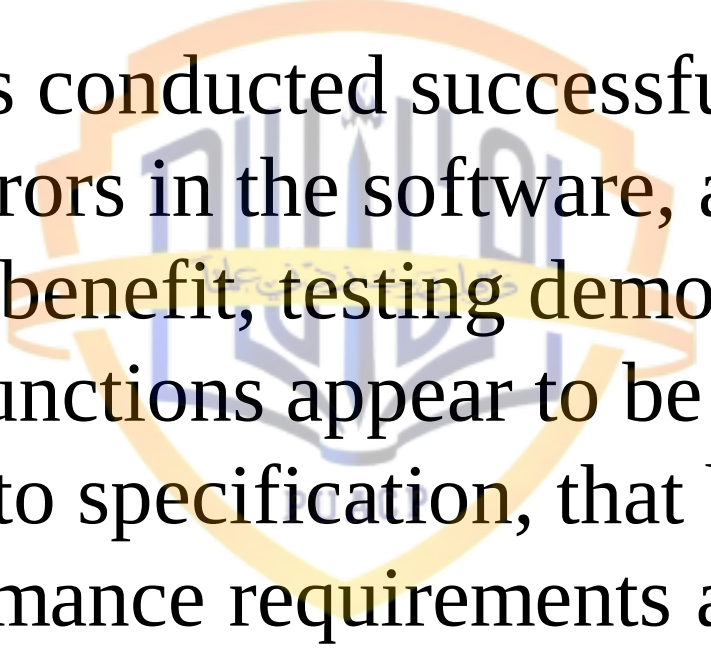
Testing and Defect Types

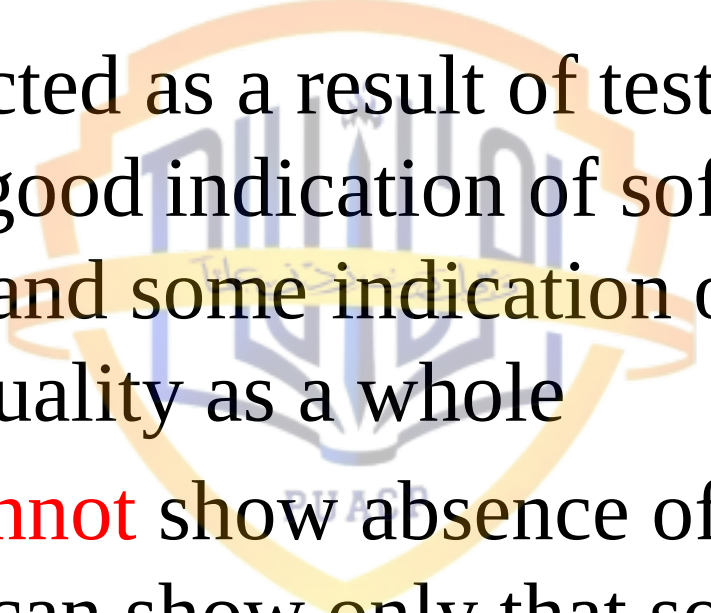
- Testing is much more effective in finding errors of commission, or things that are done wrong, than it is in finding errors of omission or things that are left out by accident

Testing Objectives

- Testing is a process of executing a program with the intent of finding an error
- A good test case is one that has a high probability of finding an as-yet-undiscovered error
- A successful test is one that uncovers an as-yet-undiscovered error

- These objectives imply a dramatic change in viewpoint. They move counter to the commonly held view that a successful test is one in which no errors are found. Our objective is to design tests that systematically uncover different classes of errors and to do so with a minimum amount of time and effort

- 
- If testing is conducted successfully, it will uncover errors in the software, and as a secondary benefit, testing demonstrates that software functions appear to be working according to specification, that behavioral and performance requirements appear to have been met

- 
- Data collected as a result of testing can provide a good indication of software reliability and some indication of the software quality as a whole
 - Testing **cannot** show absence of errors and defects, it can show only that software errors and defects are present

Testing Principles - 1

- All tests should be traceable to customer requirements
 - The objective of software testing is to find defects. It follows that the most severe defects are those that cause systems to fail to meet their requirements
- Tests should be planned long before testing begins
 - Test planning can begin as soon as the requirements model is complete

Testing Principles - 2

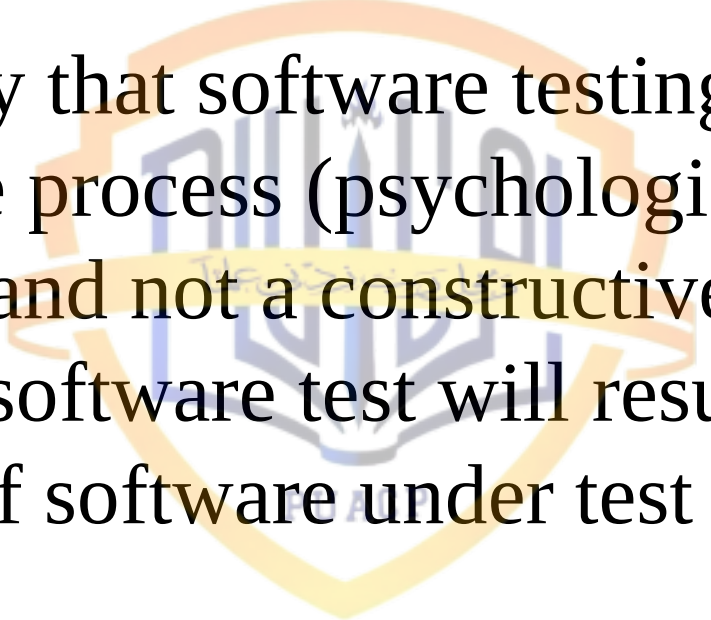
- The Pareto (80-20) principle applies to software testing
 - 80% of all defects found will likely be traced to 20% of modules. These error-prone modules should be isolated and tested thoroughly
- The testing should begin “in the small” and progress toward testing “in the large”
 - The first tests planned and executed focus on individual components. As testing progresses, focus shifts to integrated clusters of components

Testing Principles - 3

- Exhaustive testing is not possible
 - The number of path permutations for even a moderately sized program is exceptionally large. It is impossible to execute every combination of paths during testing. It is possible, however, to adequately cover program logic and to ensure that all conditions in the component-level design have been exercised

Testing Principles - 4

- To be most effective, testing should be conducted by an independent third party
 - There are many testing techniques, as we will discuss them, and the software engineer who created the software system is not the best person to conduct all tests for the software. For certain kinds of testing, the best results are obtained by conducting these tests by professional testers

- 
- We can say that software testing is a destructive process (psychologically speaking) and not a constructive process, because a software test will result in the breaking of software under test



Following slide to be inserted

Attributes of a Good Test

Attributes of a Good Test

- A good test has a high probability of finding an error
- A good test is not redundant
- A good test should be “best of breed”
- A good test should be neither too simple nor too complex

A good test has a high probability of finding an error

- The tester must understand the software and attempt to develop a mental picture of how the software might fail
- Ideally, the classes of failure are probed and a set of tests should be designed to show that software fails in a particular situation

A good test is not redundant

- Testing time and resources are limited and there is no point in conducting a test that has the same purpose as another test
- Every test should have a different purpose, even if it subtly different

A good test should be ‘best of breed’

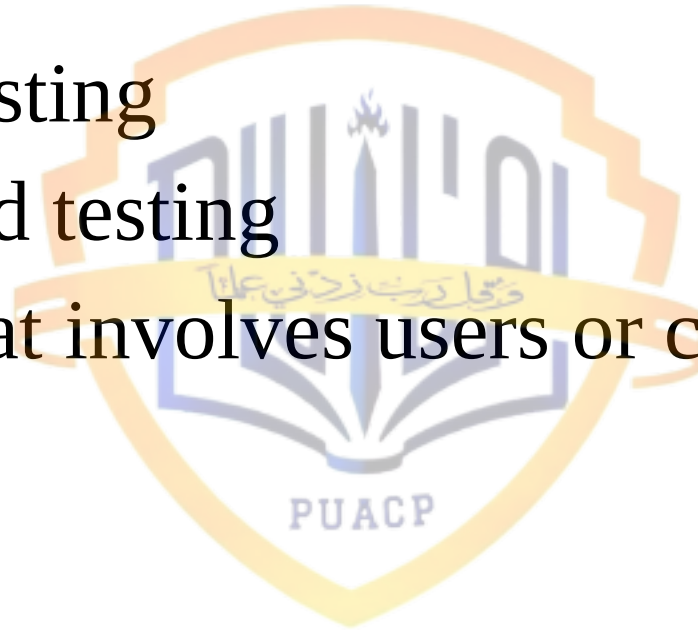
- In a group of tests that have a similar intent, time and resource limitations may force us to execute only a subset of these tests. In such cases, the tests that has the highest likelihood of uncovering a whole class of errors should be used

A good test should be neither too simple nor too complex

- Ideally, each test should be executed separately, instead of being combined with many tests
- If tests are combined into one test, this can result in masking of errors

Broad Categories of Testing

- General testing
- Specialized testing
- Testing that involves users or clients

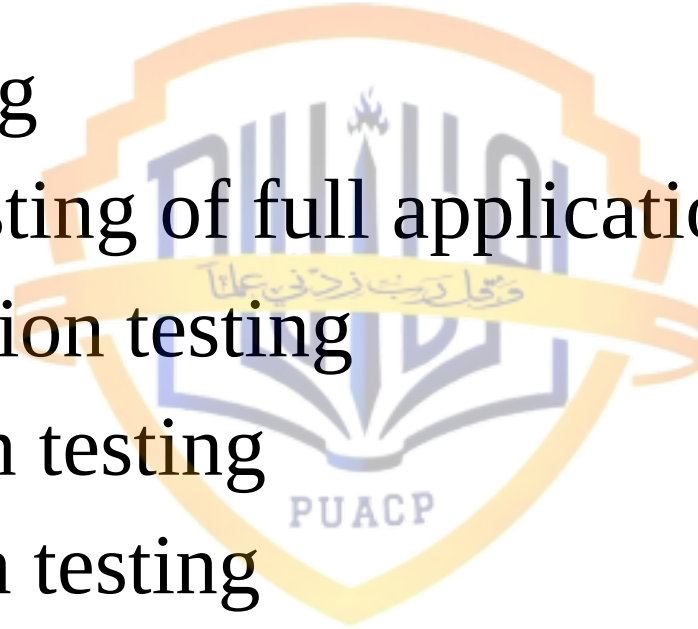


General Forms of Testing

- Concerned with almost any kind of software and seek to eliminate common kinds of bugs such as branching errors, looping errors, incorrect outputs, and the like

Examples of General Forms of Testing

- Unit testing
- System testing of full application
- New function testing
- Regression testing
- Integration testing

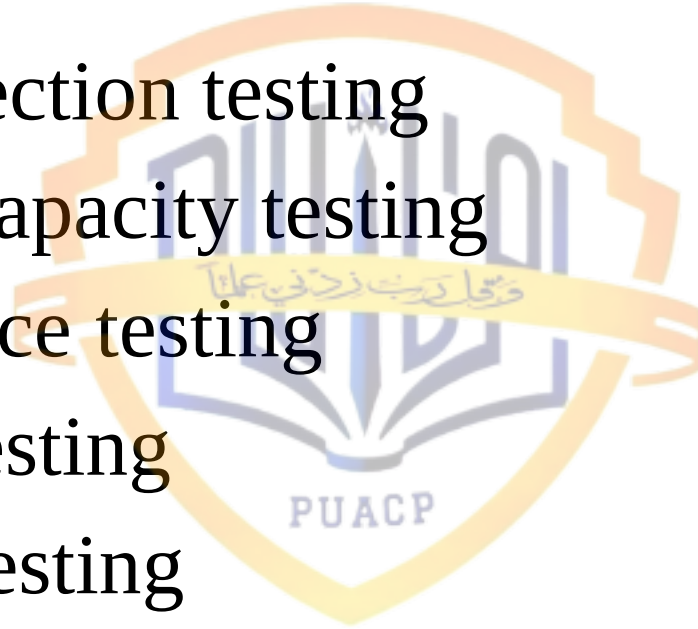


Specialized Forms of Testing

- More narrow in focus and seek specific kinds of errors such as problems that only occur under full load, or problems that might slow down performance

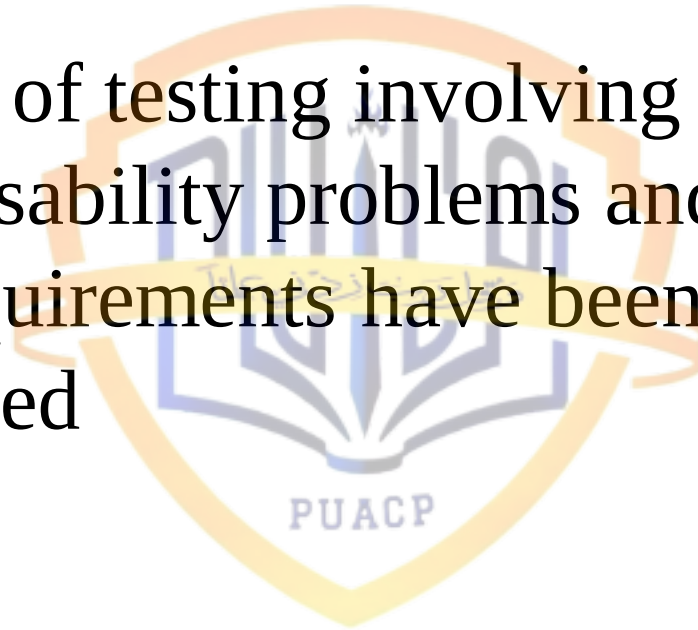
Examples of Specialized Forms of Testing

- Viral protection testing
- Stress or capacity testing
- Performance testing
- Security testing
- Platform testing
- Year 2000 testing was also an example of this
- Independent testing



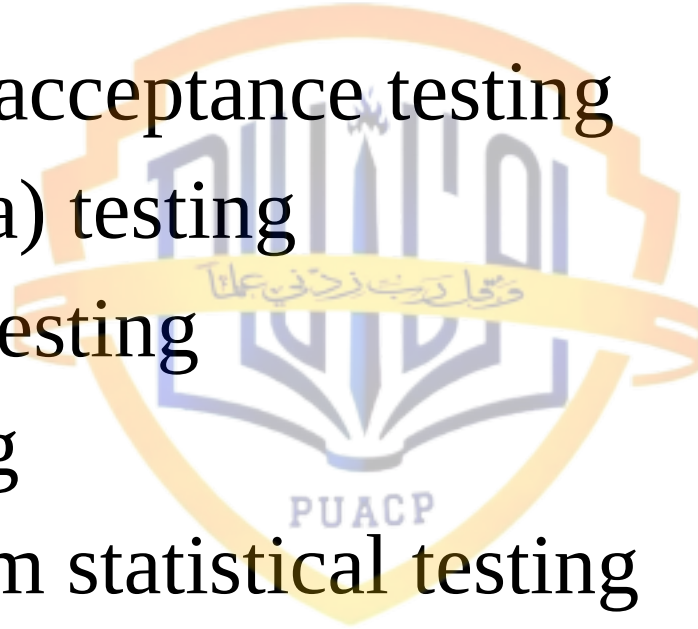
Forms of Testing Involving Users

- The forms of testing involving users are aimed at usability problems and ensuring that all requirements have been in fact implemented



Examples of Forms of Testing Involving Users

- Customer acceptance testing
- Field (Beta) testing
- Usability testing
- Lab testing
- Clean-room statistical testing



Test Case Design - 1

- Any engineered product can be tested in one of two ways
 - Knowing the specified function that a product has been designed to perform
 - Knowing the internal workings of a product

Test Case Design - 2

- In the first case, tests can be conducted that demonstrate each function is fully operational while at the same time searching for errors in each function
- In the second case, tests can be conducted to ensure that internal operations are performed according to the specifications and all internal components have been adequately exercised

Test Case Design - 3

- In the first case, testing is focused on the external behavior of a software system or its various components, and we cannot see inside the components
- While in the second case, testing is focused on the internal implementation, and we must see inside the component

Testing Techniques

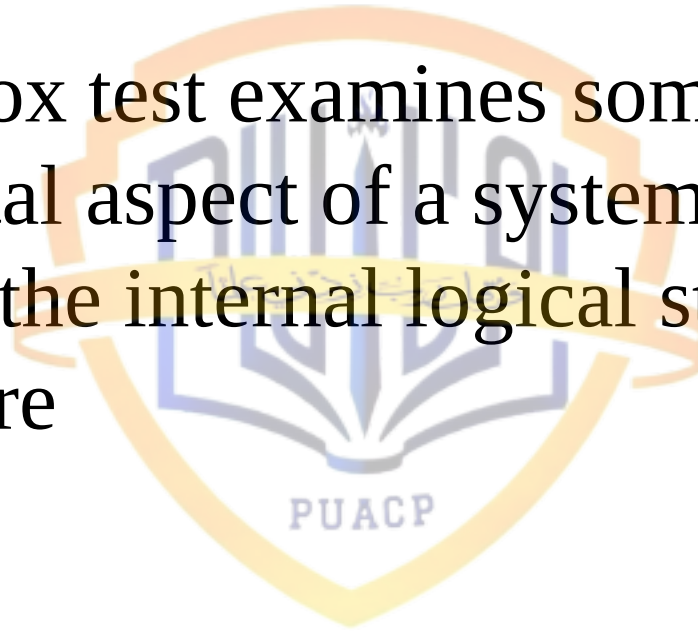
- Black-box testing (BBT)
 - aka functional/behavioral testing
- White-box testing (WBT)
 - aka structural/glass-box testing

Black-Box Testing - 1

- Black-box testing alludes to tests that are conducted at the software interface
- Although they are designed to uncover errors, they are also used to demonstrate that software functions are operational, that input is properly accepted and output is correctly produced, and that the integrity of external information is maintained

Black-Box Testing - 2

- A block-box test examines some fundamental aspect of a system with little regard for the internal logical structure of the software



Black-Box Testing - 3

- The inner structure or control flow of the application is not known or viewed as irrelevant for constructing test cases. The application is tested against external specifications and/or requirements in order to ensure that a specific set of input parameters will in fact yield the correct set of output values

Black-Box Testing - 4

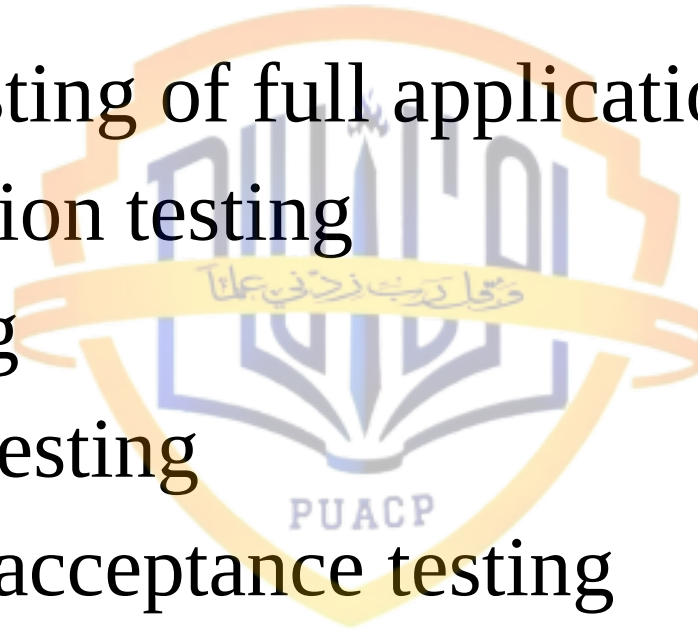
- It is useful for ensuring that the software more or less is in concordance with the written specifications and written requirements
- The simplest form of BBT is to start running the software and make observations in the hope that it is easy to distinguish between expected and unexpected behavior

Black-Box Testing - 5

- This is ad-hoc testing and it is easy to identify some unexpected behavior, like system crash
- With repeated executions, we can determine the cause to be related to software and then pass that information to the people responsible for repairs

Black-Box Testing Approaches

- System testing of full application
- New function testing
- Lab testing
- Usability testing
- Customer acceptance testing
- Field (Beta) testing
- Clean-room statistical testing



White-Box Testing - 1

- White-box testing of software is predicated on close examination of procedural detail
- Logical paths through the software are tested by providing test cases that exercise specific sets of conditions and/or loops
- The “status of the programs” may be examined at various points to determine if the expected/asserted status corresponds to the actual status

White-Box Testing - 2

- The test developer is privy to inner structure of the application and knows the control flow through the application, or at least knows the control if the software works correctly
- It is useful for ensuring that all or at least most paths through the applications have been executed in the course of testing

White-Box Testing - 3

- Using white-box testing methods, software engineers can derive test cases that
 - Guarantee that all independent paths within a module have been exercised at least once
 - Exercise all logical decisions on their true and false sides
 - Execute all loops at their boundaries and within their operational bounds
 - Exercise internal data structures to ensure their validity

White-Box Testing - 4

- The simplest form of WBT is statement coverage testing through the use of various debugging tools, or debuggers, which help us in tracing through program executions
- By doing so, the tester can see if a specific statement has been executed, and if the result or behavior is expected

White-Box Testing - 5

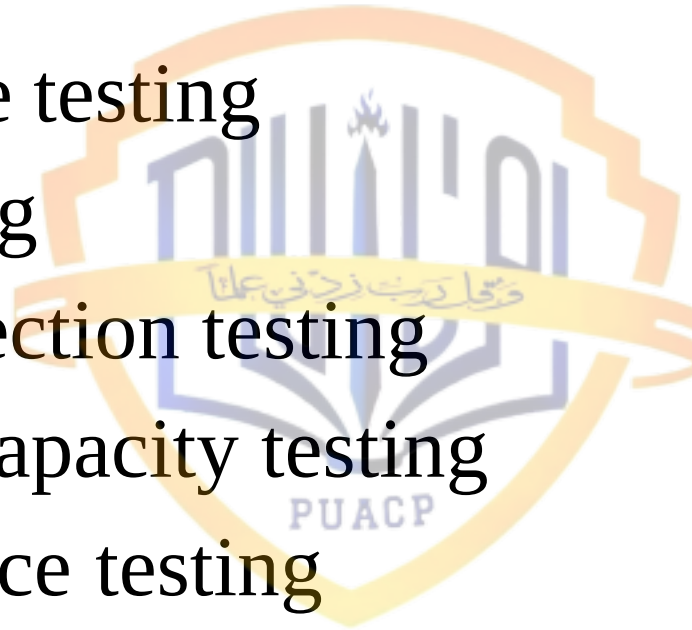
- One of the advantages is that once a problem is detected, it is also located
- However, problems of omission or design problems cannot be easily detected through white-box testing, because only what is present in the code is tested
- Another important point is that the tester needs to be very familiar with the code under testing to trace through its executions

White-Box Testing - 6

- Therefore, typically white-box testing is performed by the programmers themselves
- We'll have some discussion on this topic, that is who is most productive for which kind of testing at a later stage in this course

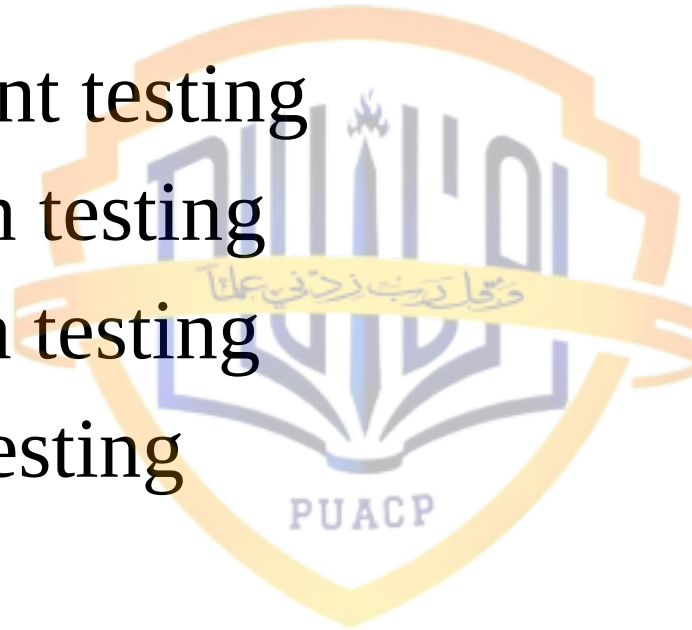
White-Box Testing Approaches

- Subroutine testing
- Unit testing
- Viral protection testing
- Stress or capacity testing
- Performance testing
- Security testing
- Year 2000 testing



Mixed Testing Approaches

- Independent testing
- Regression testing
- Integration testing
- Platform testing

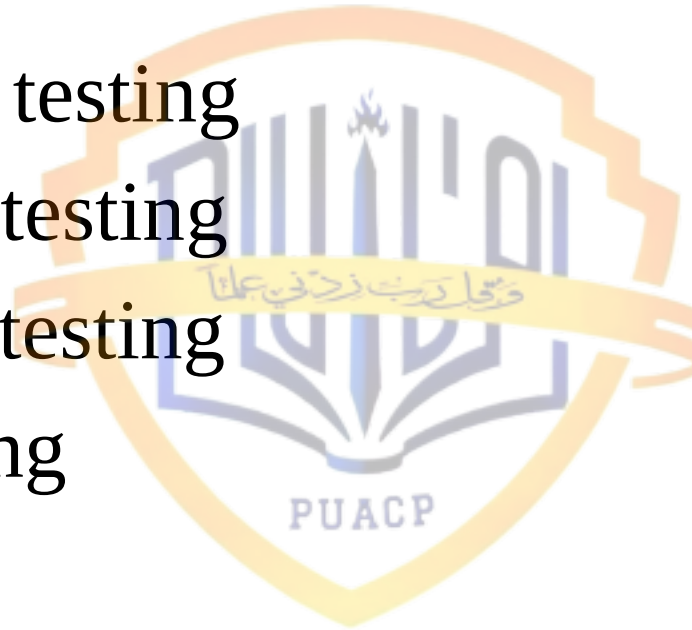


Comparing BBT with WBT

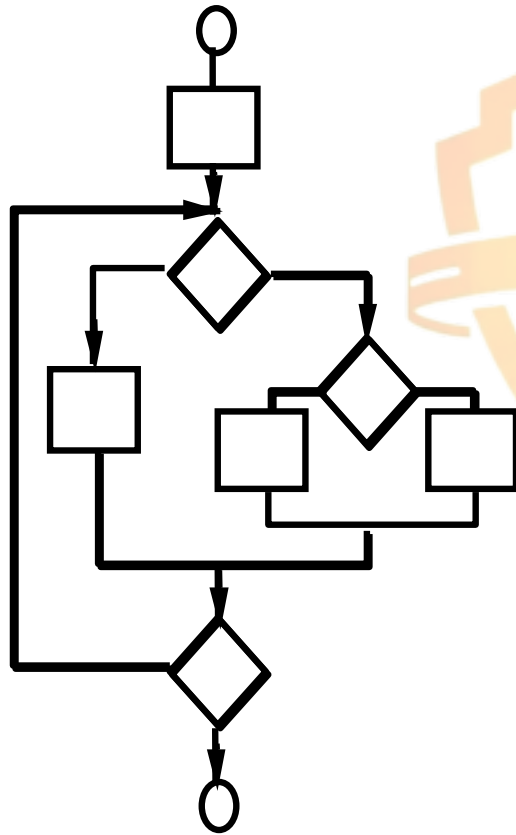
- It is the perspective, which primarily distinguishes black-box testing from white-box testing
- However, BBT and WBT can be compared using other criteria also
 - Objects
 - Timeline
 - Defect focus
 - Defect detection and fixing
 - Techniques
 - Testers

White-Box Testing Techniques

- Basis path testing
- Condition testing
- Data flow testing
- Loop testing



Cyclomatic Complexity



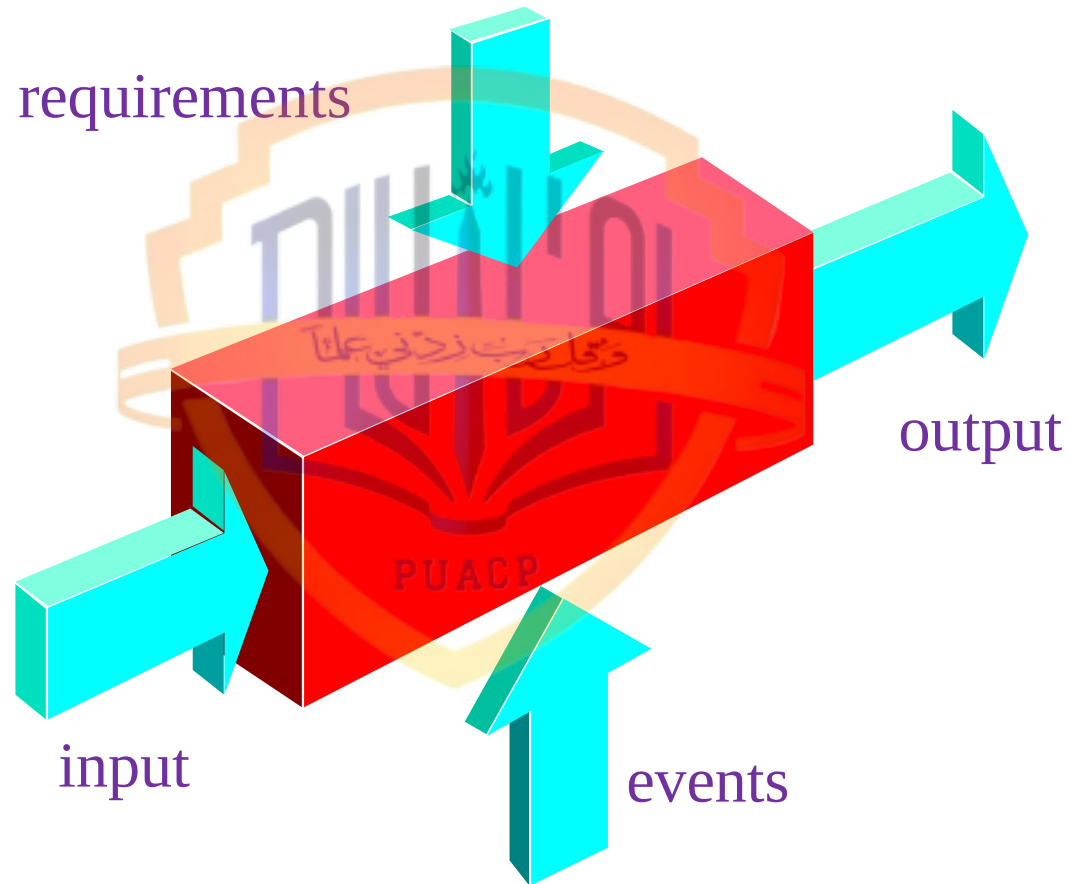
Number of simple decisions + 1

or

Number of enclosed areas + 1

In this case, $V(G) = 4$

Black-Box Testing

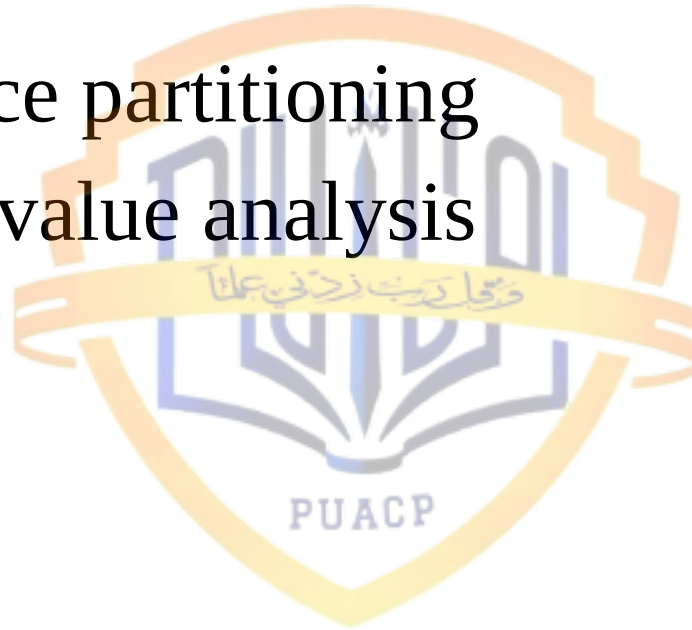


Black-box Testing - 2

- BBT attempts to find errors in the following categories
 - Incorrect or missing functions
 - Interface errors
 - Errors in data structures or external data base access
 - Behavior or performance errors
 - Initialization and termination errors

BBT Techniques

- Equivalence partitioning
- Boundary value analysis



Characteristics of Testing Strategies - 1

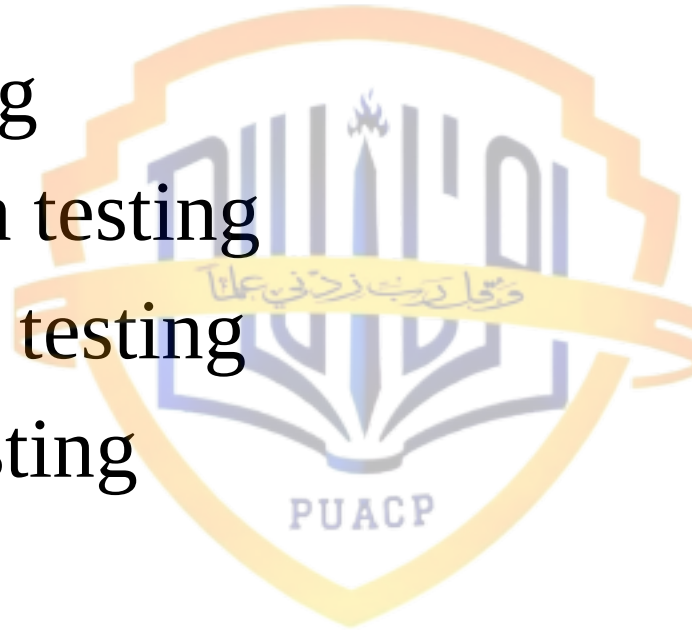
- (1) Testing begins at the component level and works “outward” toward the integration of the entire computer-based system
- (2) Different testing techniques are appropriate at different points in time

Characteristics of Testing Strategies - 2

- (3) Testing is conducted by the developer of the software and (for large projects) an independent test group
- (4) Testing and debugging are different activities, but debugging must be accommodated in any testing strategy

Testing Strategy - 1

- Unit testing
- Integration testing
- Validation testing
- System testing



Testing Strategy: Unit Testing

- Unit testing makes heavy use of white-box testing techniques, exercising specific paths in a module's control structure to ensure complete coverage and maximum error detection

Testing Strategy: Integration Testing

- Next, components must be assembled or integrated to form the complete software package
- Integration testing addresses the issues associated with the dual problems of verification and program construction
- BB test case design techniques are the most prevalent during integration

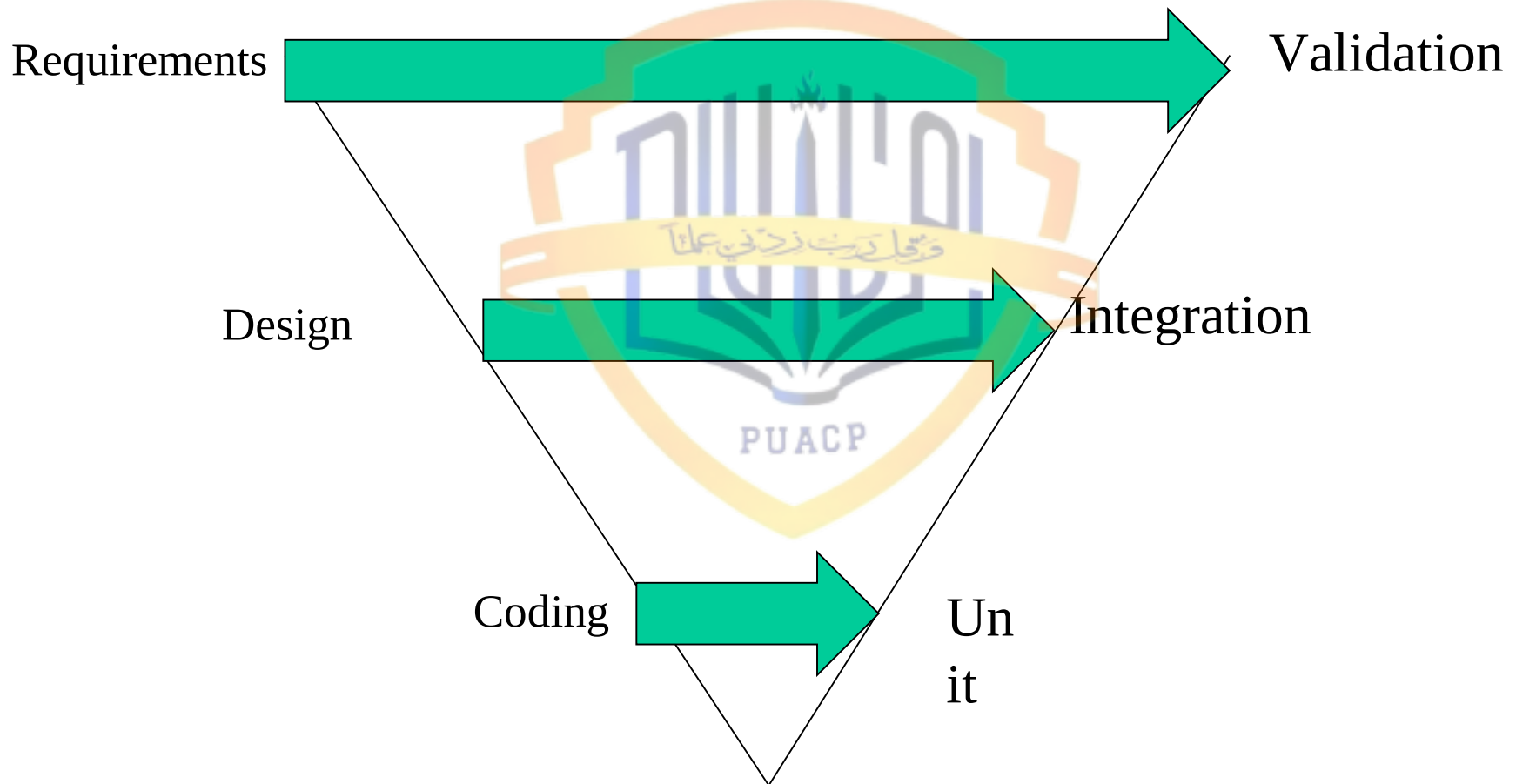
Testing Strategy: Validation Testing

- After software has been integrated, a set of high-order tests are conducted
- Validation criteria (established during requirements analysis) must be tested
- Validation testing provides final assurance that software meets all functional, behavioral, and performance requirements
- BBT techniques are used exclusively here

Testing Strategy: System Testing

- System testing is the last high-order testing and it falls outside the boundaries of software engineering and into the broader context of computer system engineering
- System testing verifies that all elements (hardware, people, databases) mesh properly and that overall system function/performance is achieved

The “V” Model of Software Testing



Unit Testing: Common Errors Found

- Among the more common errors in computation are
 - Misunderstood or incorrect arithmetic precedence
 - Mixed mode operations
 - Incorrect initialization
 - Precision inaccuracy
 - Incorrect symbolic representation of expression

Unit Testing: Test Cases

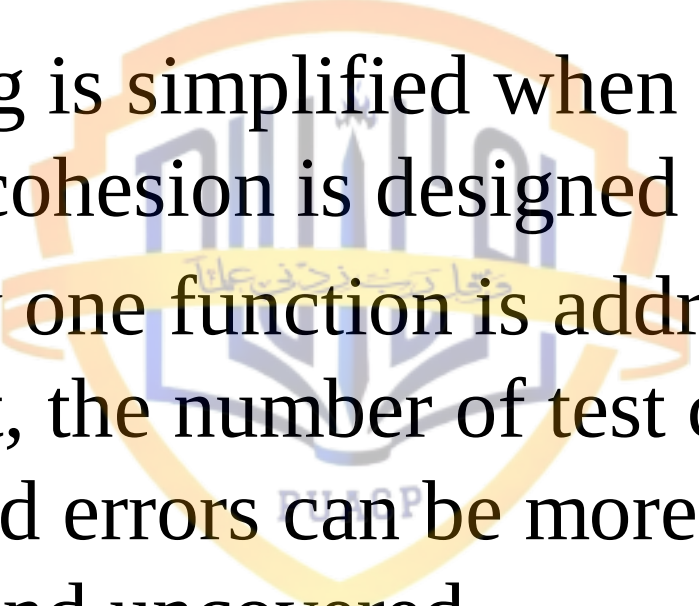
- Test cases should uncover errors such as
 - Comparison of different data types
 - Incorrect logical operators or precedence
 - Expectation of equality when precision error makes equality unlikely
 - Incorrect comparison of variables
 - Improper or non-existent loop termination
 - Failure to exit when divergent iteration is encountered
 - Improperly modified loop variables

Unit Testing: Error Handling

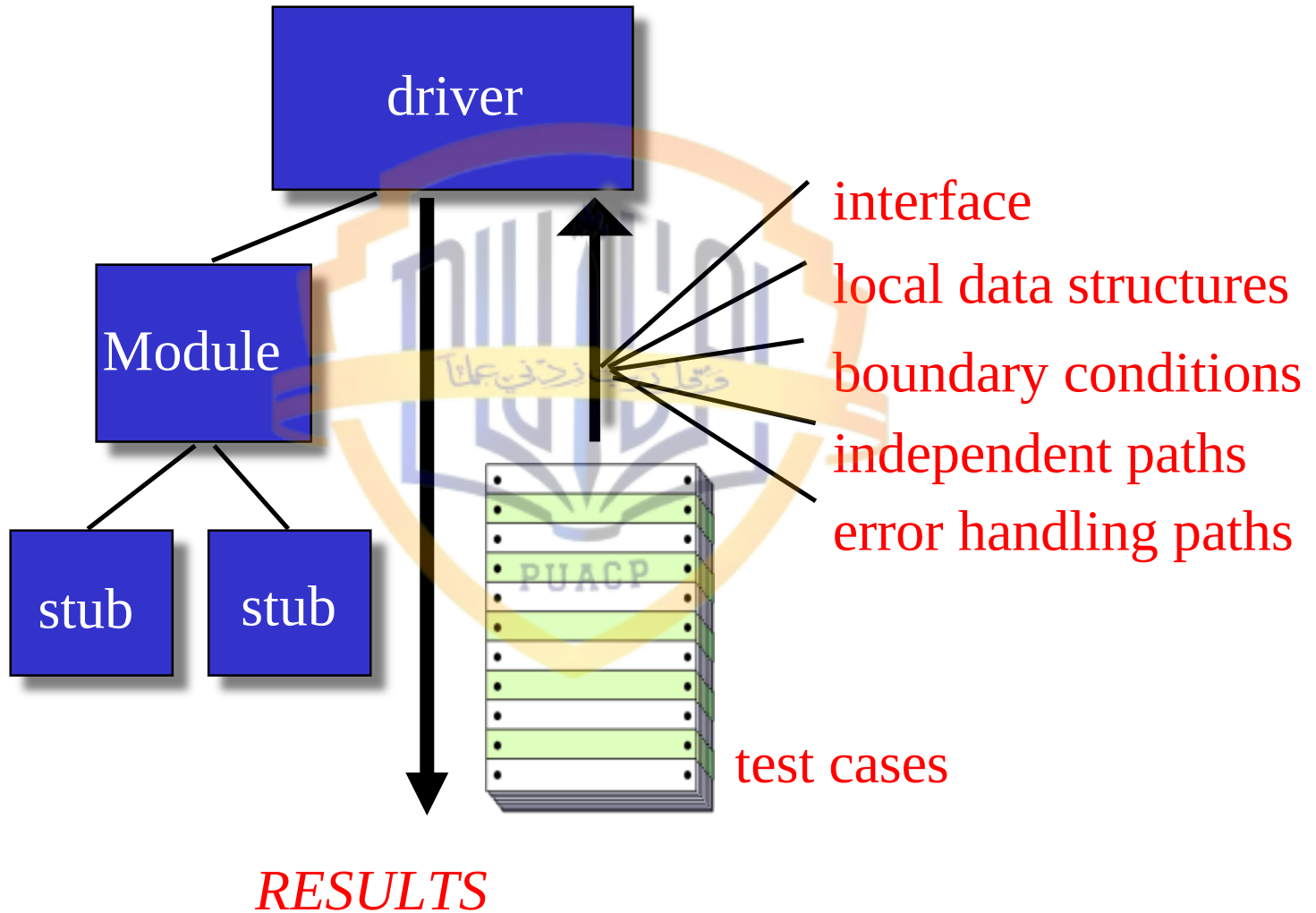
- Among the potential errors that should be tested when error handling is evaluated are
 - Error description is unintelligible
 - Error noted does not correspond to error encountered
 - Error condition causes system intervention prior to error handling
 - Exception-condition processing is incorrect
 - Error description does not provide enough information to assist in the location of the cause of the error

Unit Testing: Boundary Testing

- Boundary testing is the last and probably the most important task of unit testing step
- Software often fails at its boundaries
- Test cases that exercise data structure, control flow, and data values just below, at, and just above maxima and minima are very likely to uncover errors

- 
- Unit testing is simplified when a component with high cohesion is designed
 - When only one function is addressed by a component, the number of test cases is reduced and errors can be more easily predicted and uncovered
 - Let's see a unit test environment

Unit Test Environment



Integration Testing - 1

- “If they all work individually, why do you doubt that they’ll work when we put them together?”
- The problem, of course, is “putting them together” – interfacing

Integration Testing - 2

- Data can be lost across an interface
- One module can have an inadvertent, adverse affect on another
- Sub-functions, when combined, may not produce the desired major function
- Individually acceptable imprecision may be magnified to unacceptable levels
- Global data structures can present problems

Software Integration



Integration Testing - 3

- Integration testing is a systematic technique for constructing the program structure while at the same time conducting tests to uncover errors associated with interfacing
- Individual components have already been unit tested and the structure is dictated by design

Integration Testing - 4

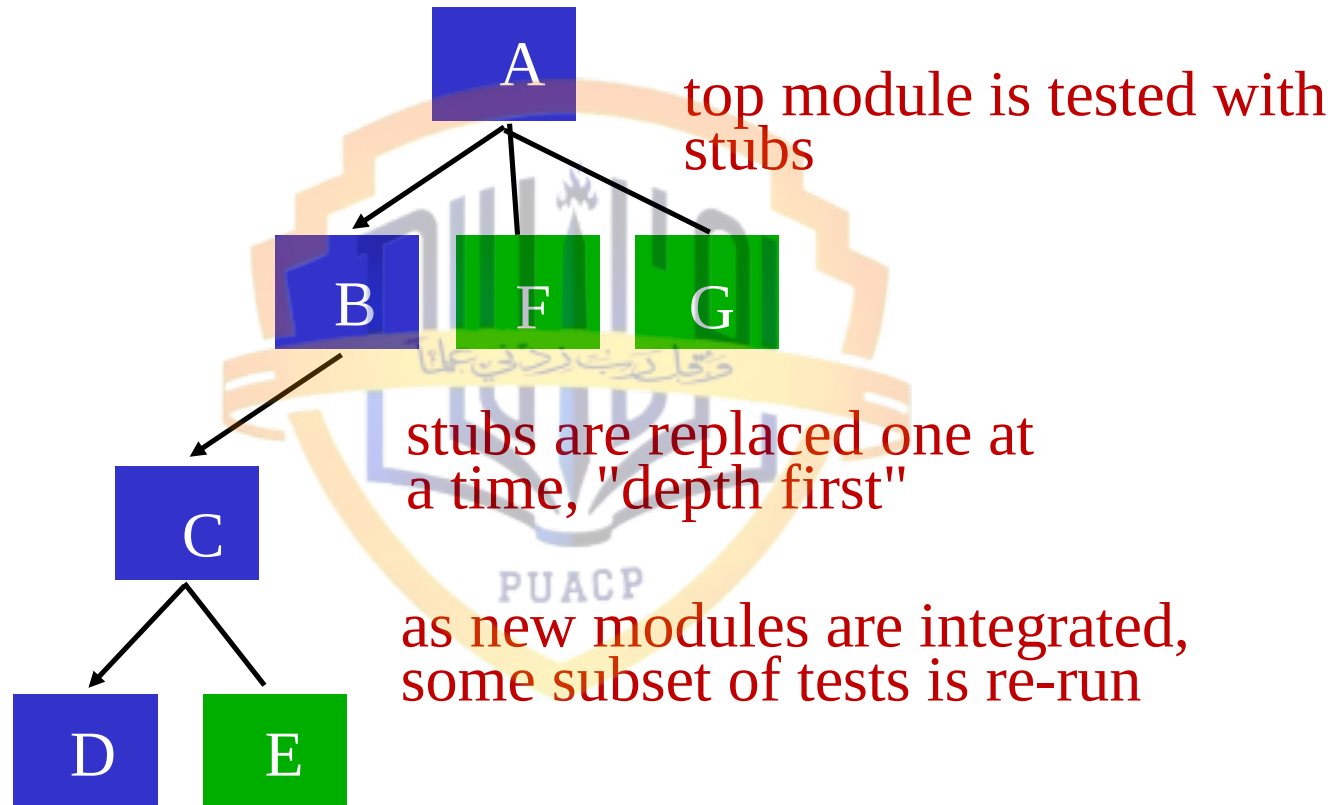
- There is tendency, sometimes, to attempt non-incremental integration – a sort of “big bang” approach
- All components are combined in advance
- The entire software is tested as a whole
- **And chaos usually results!**
- We should rather use incremental integration

Integration Testing Approaches

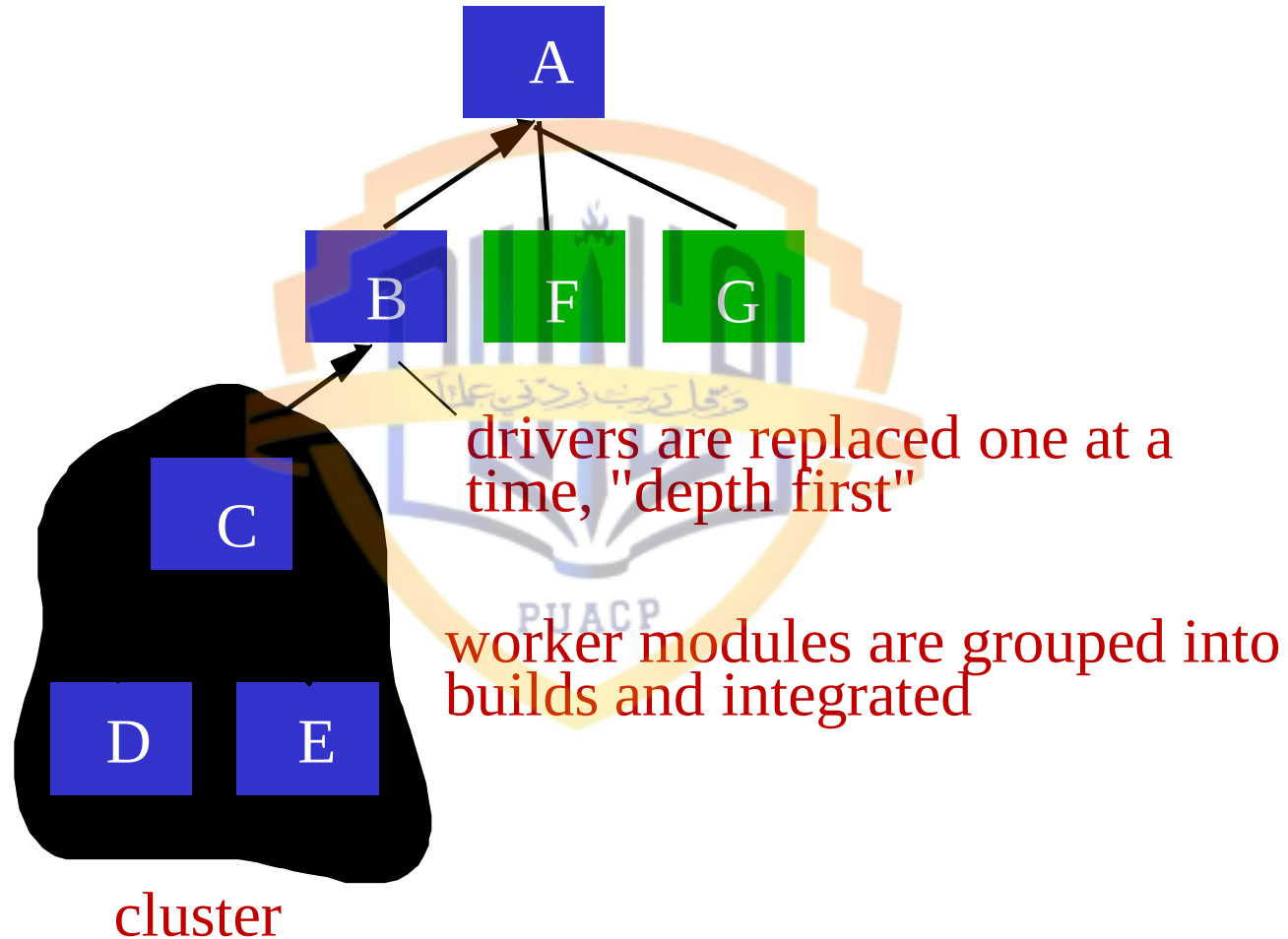
- Top-down integration
- Bottom-up integration
- Sandwich testing and integration



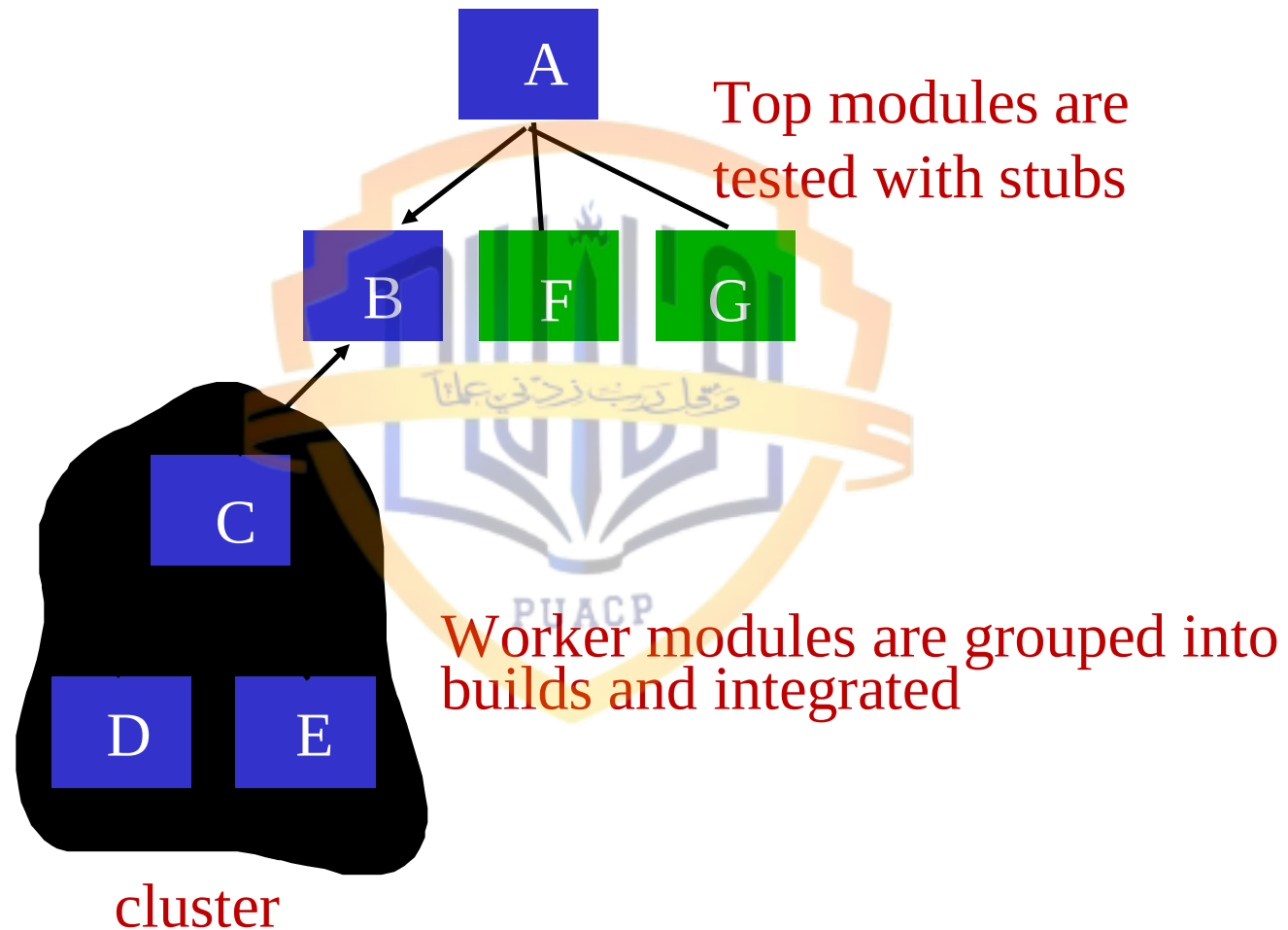
Top Down Integration



Bottom-Up Integration



Sandwich Testing and Integration



- Regression testing



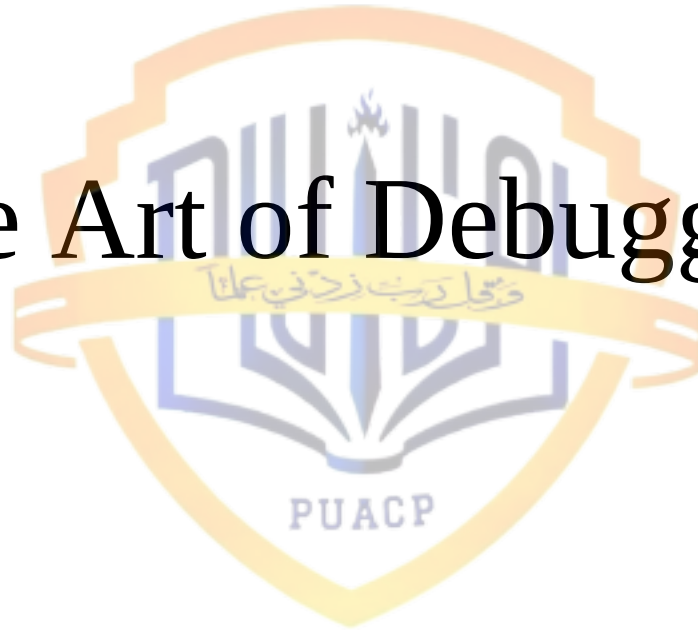
Validation Testing



Testing and Litigation of Poor Software Quality

	Reliable Software	Software Involved in Litigation for Poor Quality
Subroutine testing	Used	Used
Unit testing	Used	Used
New function testing	Used	Rushed or omitted
Regression testing	Used	Rushed or omitted
Integration testing	Used	Used
System testing	Used	Rushed or omitted
Performance testing	Used	Rushed or omitted
Capacity testing	Used	Rushed or omitted

The Art of Debugging



The Debugging Process

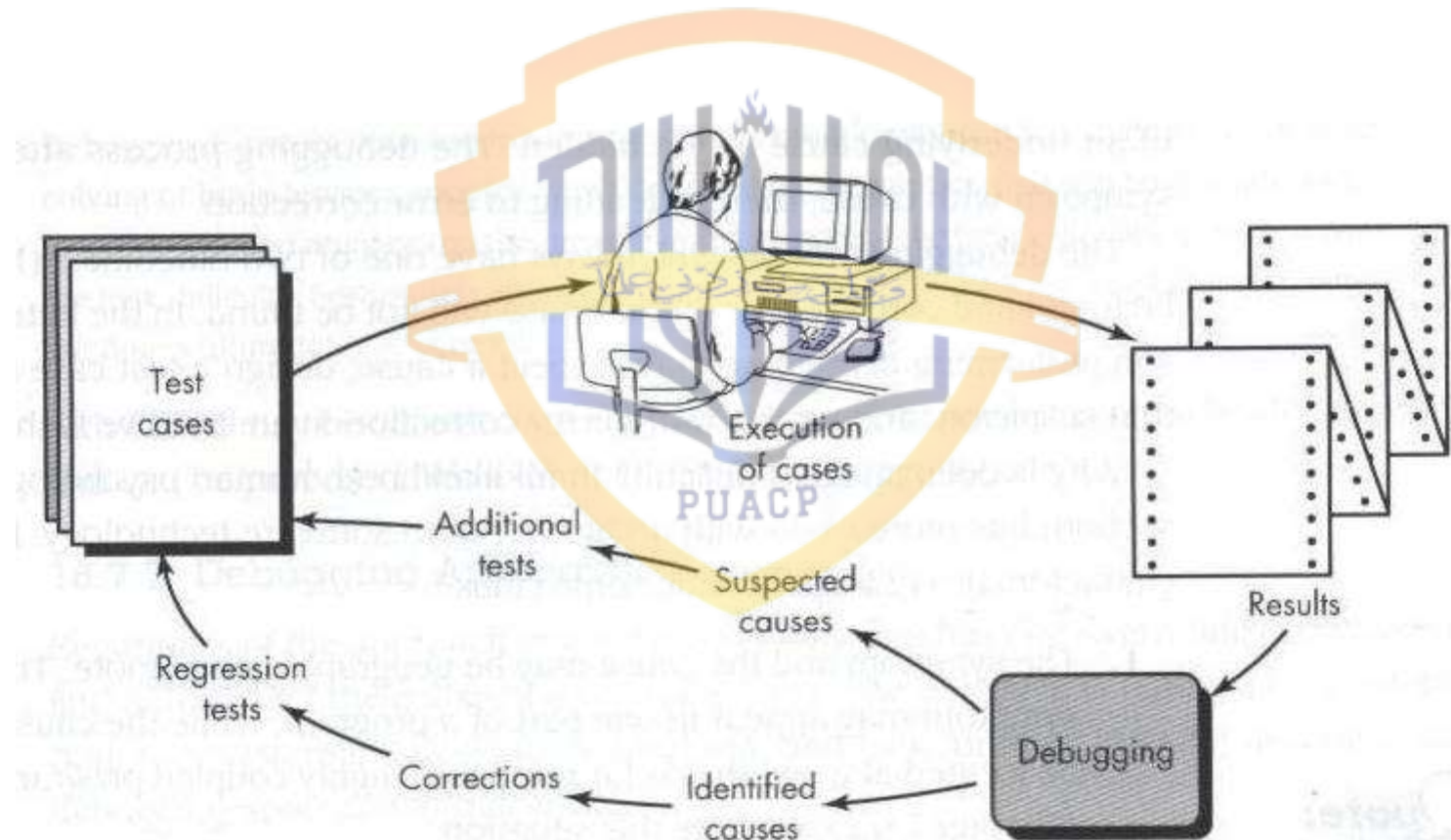


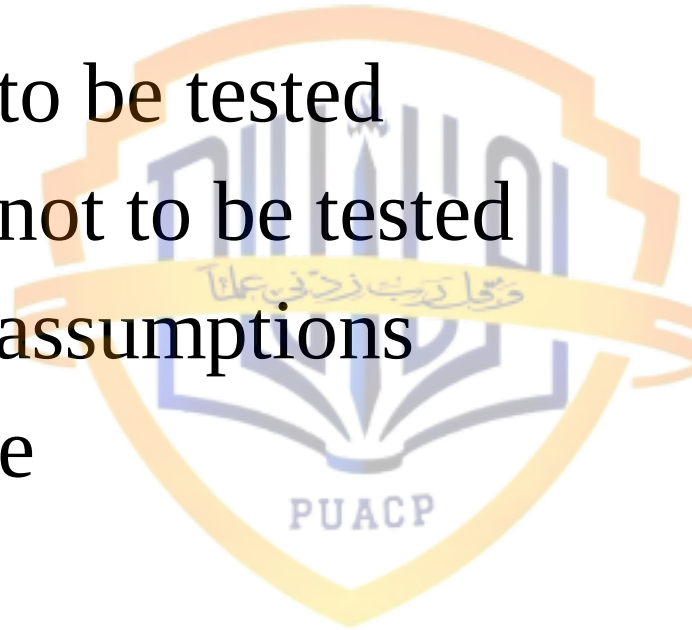
Table of Contents of A Sample Test Plan - 1

- Introduction
- Scope
- Test plan strategy
- Test environment
- Schedule
- Control procedures
- Control activities



Table of Contents of A Sample Test Plan - 2

- Functions to be tested
- Functions not to be tested
- Risks and assumptions
- Deliverable
- Test tools
- Approvals
- Exit criteria

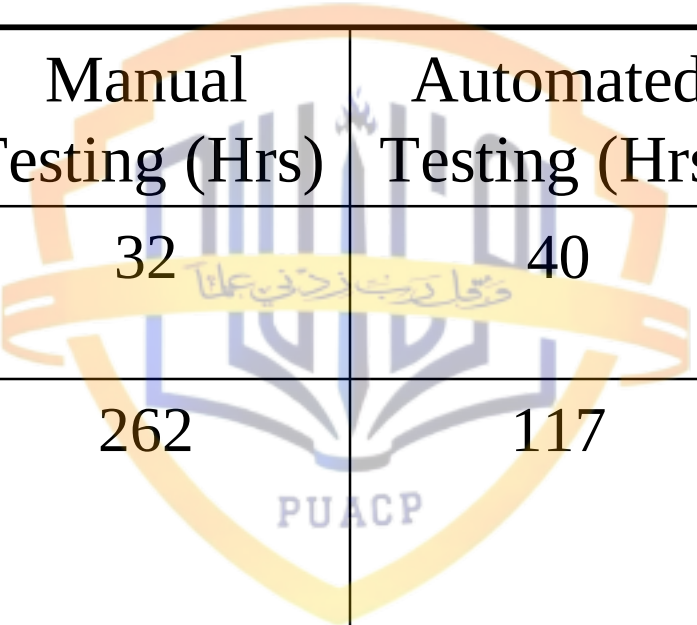


The logo of the University of Al-Qadisiyah is a shield-shaped emblem. It features a stylized open book at the base, with a flame or torch rising from its center. Above the book, there is a banner with Arabic calligraphy. The entire emblem is enclosed within a shield-like border with a serrated top edge. The colors are primarily blue, yellow, and orange.

Automated Testing

Discuss in detail

Manual Versus Automated Testing - 1



Test Steps	Manual Testing (Hrs)	Automated Testing (Hrs)	Percentage Improvement
Test Plan Development	32	40	-25%
Test Procedure Development	262	117	55%
Test Execution	466	23	95%
Test Result Analysis	117	58	50%

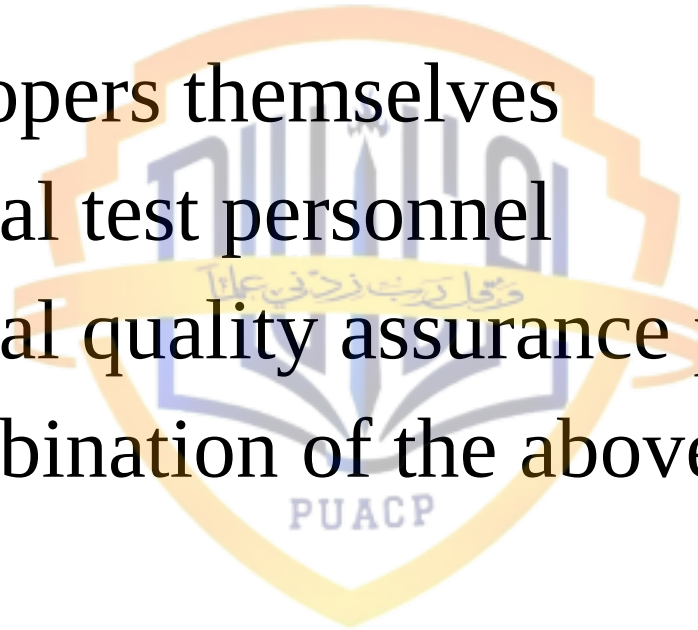
Manual Versus Automated Testing -

2

Test Steps	Manual Testing (Hrs)	Automated Testing (Hrs)	Percentage Improvement
Error Status / Correction Monitoring	117	23	80%
Report Creation	96	16	83%
Total Duration	1090	277	75%

Who Should Do Testing?

- The developers themselves
- Professional test personnel
- Professional quality assurance personnel
- Some combination of the above three



When to Stop Testing?

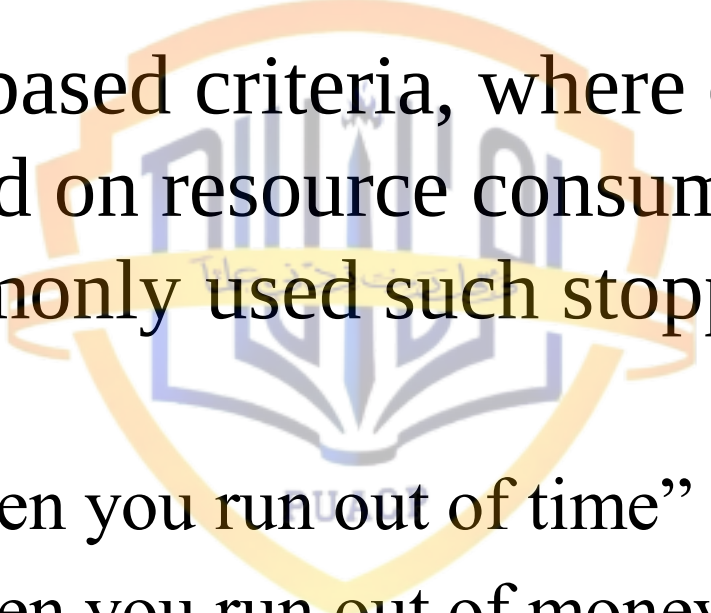


Criteria for Completion of Testing

- You're never done testing, the burden simply shifts from you to your customer
- You're done testing when you run out of time or you run out of money
- There is no definitive answer

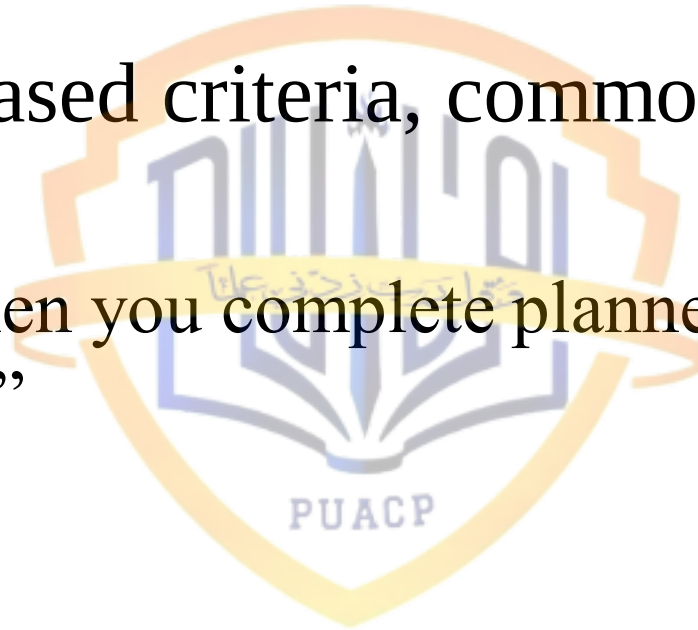
When to Stop Testing?

- On a small or local scale, we can ask: “When to stop testing for a specific test activity?”
- On a global scale, we can ask: “When to stop all the major test activities?” because the testing phase is usually the last major development phase before product release, this question is equivalent to: “When to stop testing and release the product?”

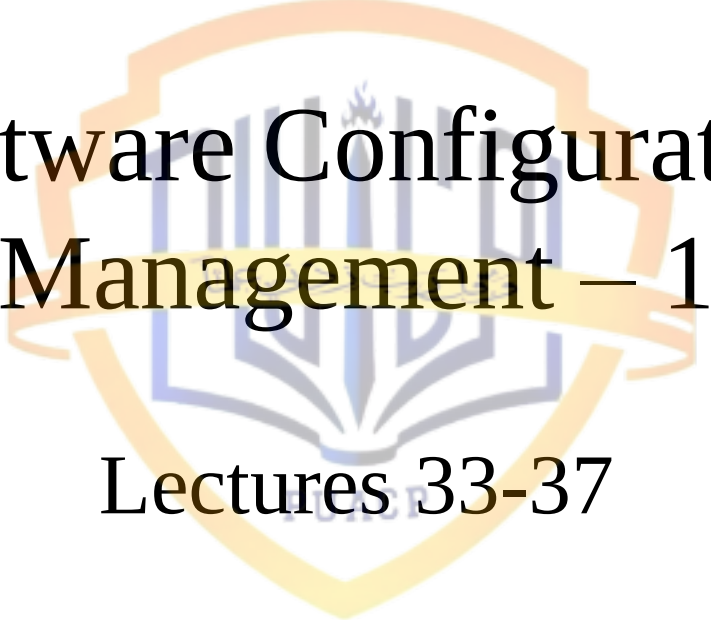
- 
- Resource-based criteria, where decision is made based on resource consumptions. The most commonly used such stopping criteria are
 - “Stop when you run out of time”
 - “Stop when you run out of money”

- Such criteria are irresponsible, as far as product quality is concerned, although they may be employed if product schedule or cost are the dominant concerns for the product in question

- Activity-based criteria, commonly in the form:
 - “Stop when you complete planned test activities”

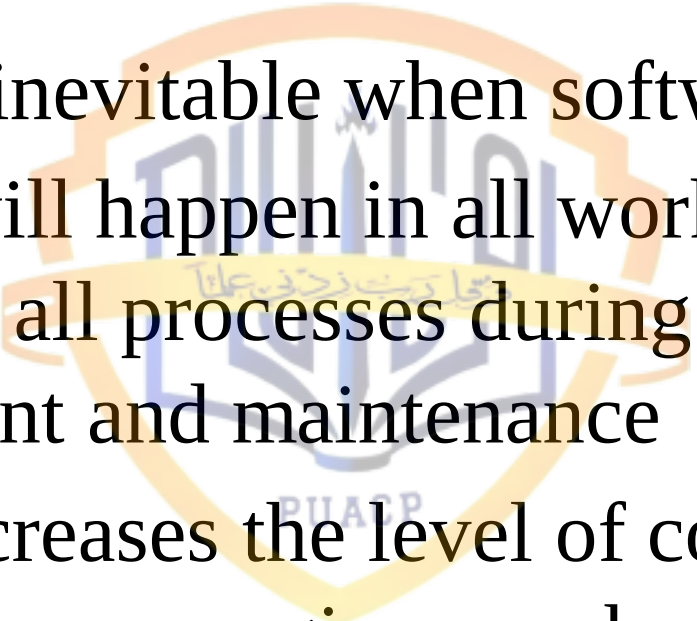


- This criterion implicitly assumes the effectiveness of the test activities in ensuring the quality of the software product. However, this assumption could be questionable without strong historical evidence based on actual data from the project concerned

The logo of the University of Jordan is a shield-shaped emblem. It features a blue open book at the base, with a blue torch rising from the center. The shield is outlined in orange and yellow. The text "Software Configuration Management – 1" is overlaid on the logo.

Software Configuration Management – 1

Lectures 33-37

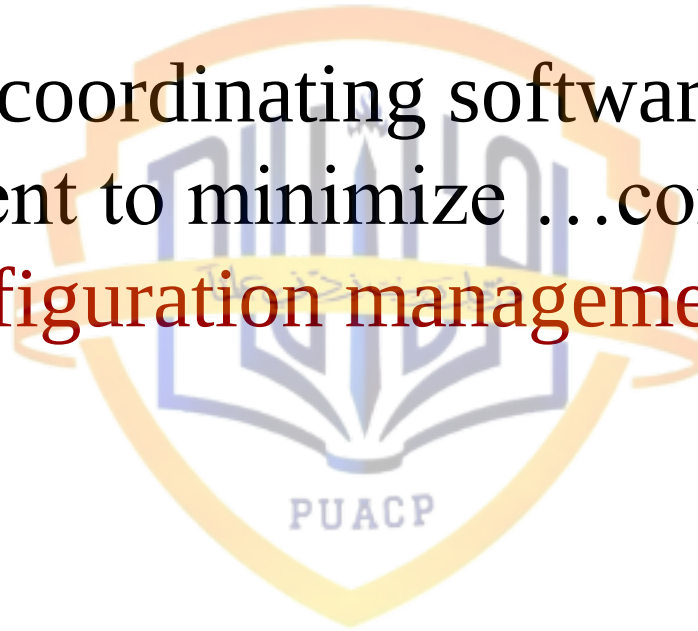
- 
- Change is inevitable when software is built
 - Changes will happen in all work products and during all processes during software development and maintenance
 - Change increases the level of confusion among software engineers who are working on a software project

Confusion

- Confusion arises when changes are not
 - Analyzed before they are made
 - Recorded before they are implemented
 - Reported to those who need to know
 - Controlled in a manner that will improve quality and reduce error
- We need to minimize this confusion, or else our projects will get out of control

Configuration Management - 1

- The art of coordinating software development to minimize ...confusion is called **configuration management**

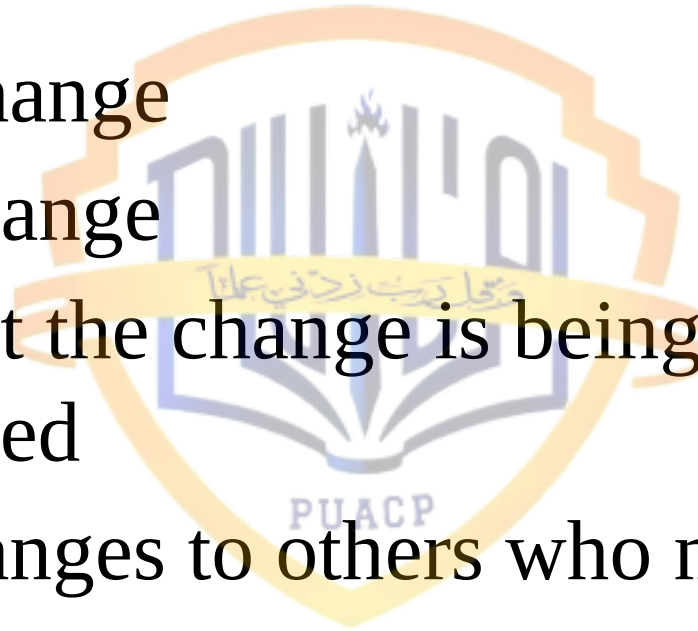


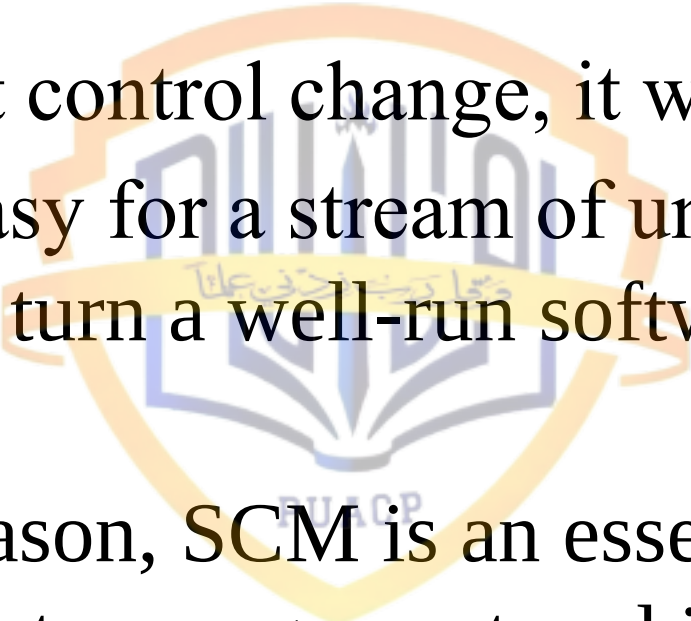
Software Configuration Management - 1

- Software configuration management (or SCM) is an umbrella activity that is applied throughout the software process
- This means that SCM is not tied to one process model or another, instead it is used in all instances of software development processes

Purpose of SCM Activities

- Identify change
- Control change
- Ensure that the change is being properly implemented
- Report changes to others who may be interested



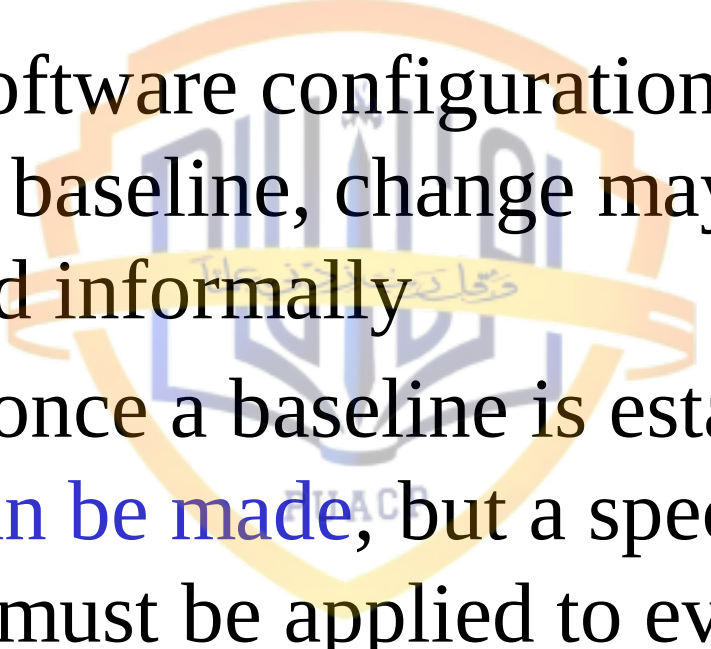
- 
- If we don't control change, it will control us
 - It's very easy for a stream of uncontrolled changes to turn a well-run software project into chaos
 - For that reason, SCM is an essential part of good project management and is a solid software engineering practice

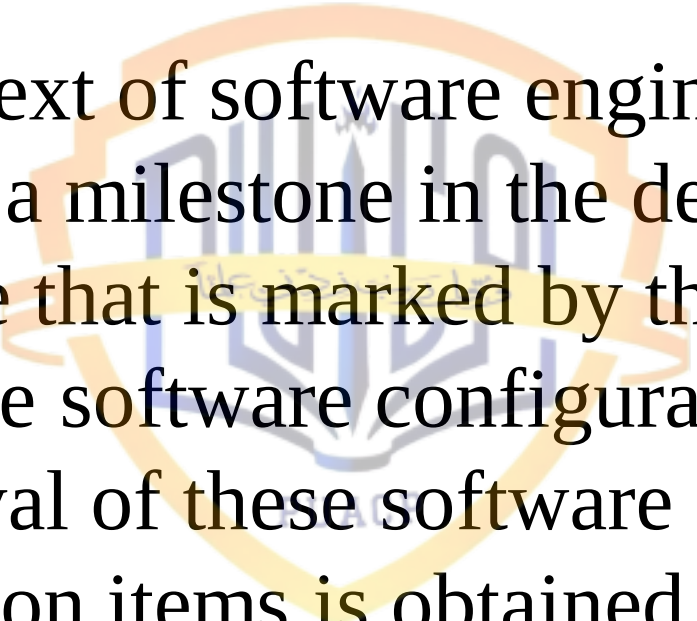
Sources of Change - 1

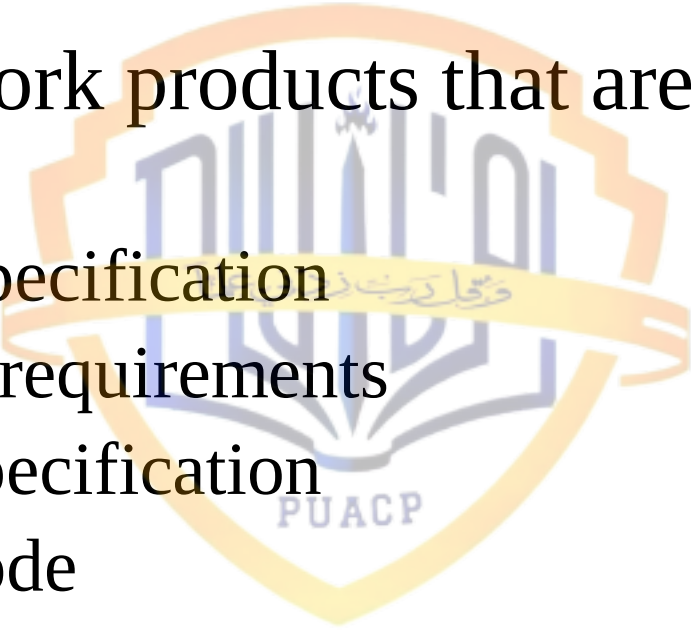
- New business or market conditions dictate changes in product requirements or business rules
- New customer needs demand modification of data produced by information systems, functionality delivered by products, or services delivered by computer-based system

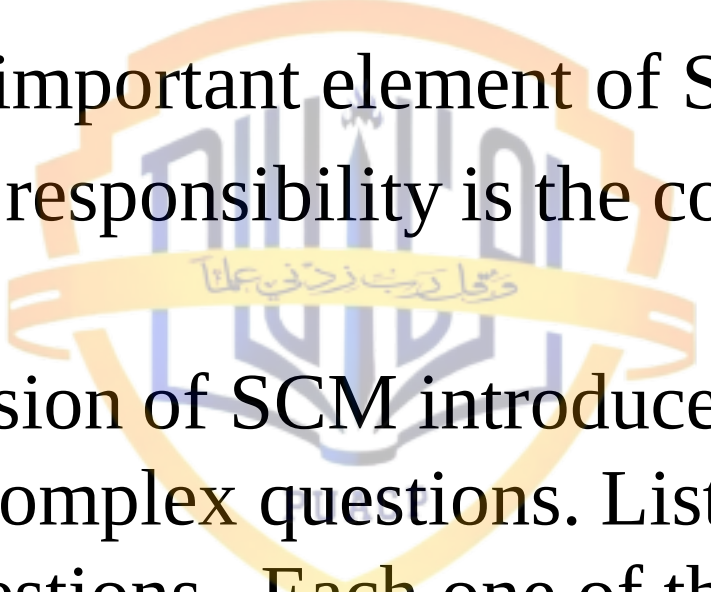
Sources of Change - 2

- Reorganization or business growth / downsizing causes changes in project priorities or software engineering team structure
- Budgetary or scheduling constraints cause a redefinition of the system or product

- 
- Before a software configuration item becomes a baseline, change may be made quickly and informally
 - However, once a baseline is established, changes can be made, but a specific, formal procedure must be applied to evaluate and verify each change request

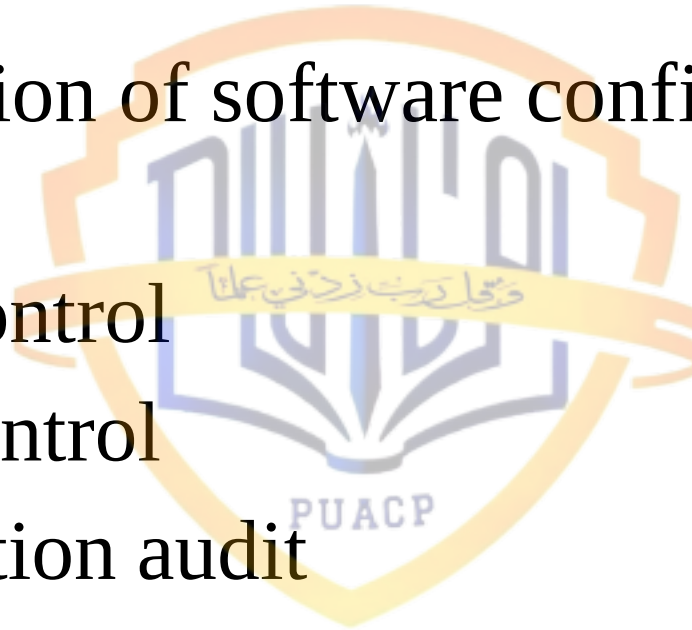
- 
- In the context of software engineering, a baseline is a milestone in the development of software that is marked by the delivery of one or more software configuration items and approval of these software configuration items is obtained through a formal technical review or inspection

- 
- The logo of the Punjab University of Agriculture, Chokkistan (PUACP) is centered in the background. It features a shield-like emblem with a yellow border. Inside the shield, there is a blue and white design that includes a stylized book or open book at the bottom, with the acronym 'PUACP' written below it. Above the book, there is Arabic calligraphy. The entire logo is semi-transparent.
- Typical, work products that are base-lined are
 - System specification
 - Software requirements
 - Design specification
 - Source code
 - Test plans/procedures/data
 - Operational system

- 
- The logo of the University of Al-Qadisiyah is a circular emblem. It features a blue shield in the center with Arabic calligraphy. Above the shield is a yellow banner with more Arabic text. The entire emblem is surrounded by a yellow border.
- SCM is an important element of SQA program
 - Its primary responsibility is the control of change
 - Any discussion of SCM introduces a set of following complex questions. Listen carefully to these questions. Each one of them is related to one or more aspect of software quality

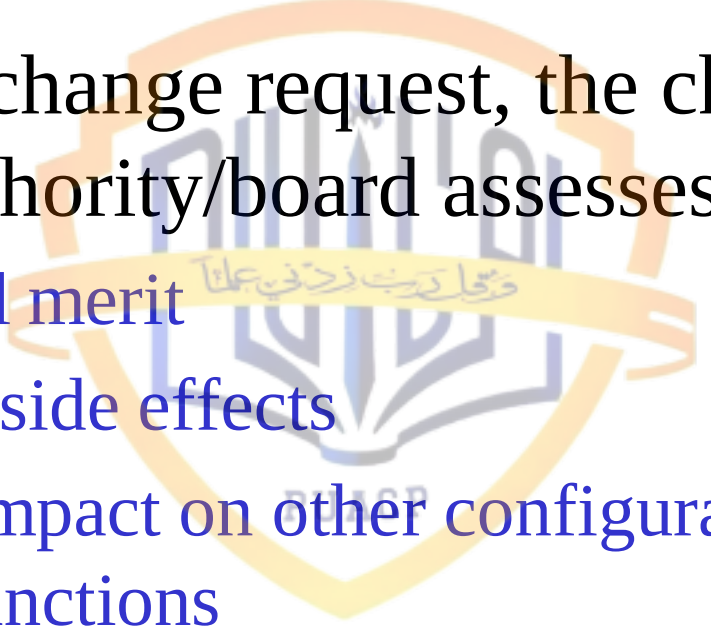
SCM Functions

- Identification of software configuration items
- Version control
- Change control
- Configuration audit
- Status accounting/reporting

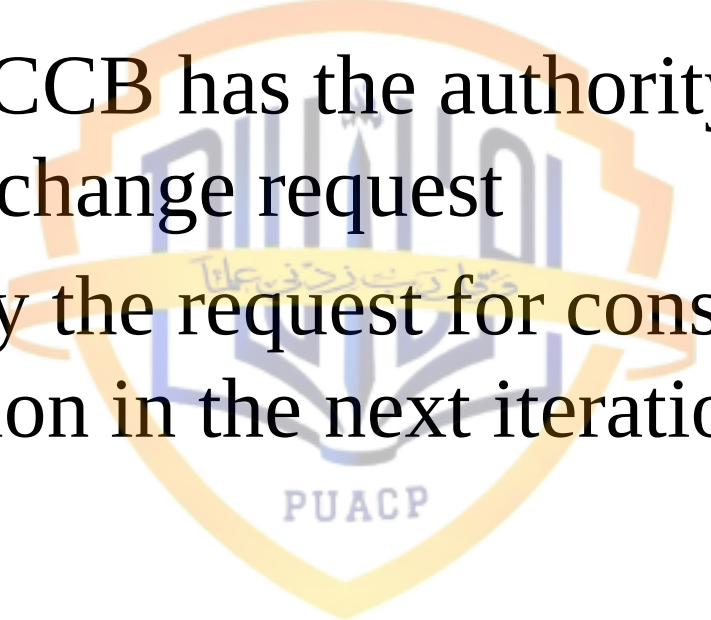


Change Control Authority/Board

- This authority is known as **change control authority** (or CCA) or **change control board** (or CCB)
- Typically, a CCA consists of representatives from software, hardware, database engineering, support, and marketing, etc., depending on the size and nature of the project

- 
- The logo of the University of Al-Qadisiyah is a circular emblem. It features a central shield with a stylized open book and a quill pen. Above the shield is a banner with Arabic calligraphy. The entire emblem is surrounded by a circular border containing the university's name in Arabic and English.
- For every change request, the change control authority/board assesses the
 - Technical merit
 - Potential side effects
 - Overall impact on other configuration items and system functions
 - Projected cost of the change

- In addition, change control authority/board may assess the impact of change beyond the SCI in question
 - How will the change affect hardware?
 - How will the change affect performance?
 - How will the change modify customer's perception of the product?
 - How will the change affect product quality and reliability?

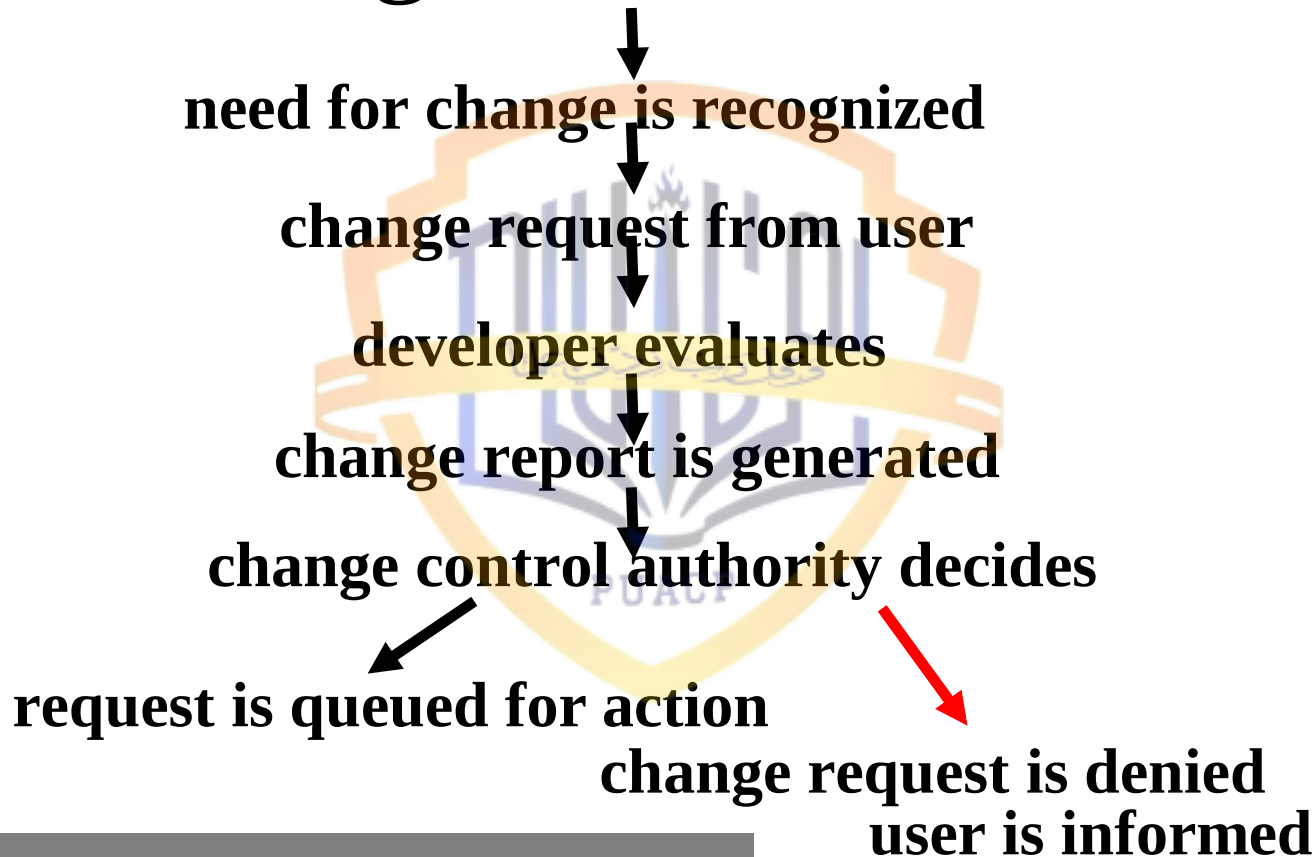
- 
- The CCA/CCB has the authority to approve or reject a change request
 - It can delay the request for consideration and inclusion in the next iteration also



Following three slides to be
inserted

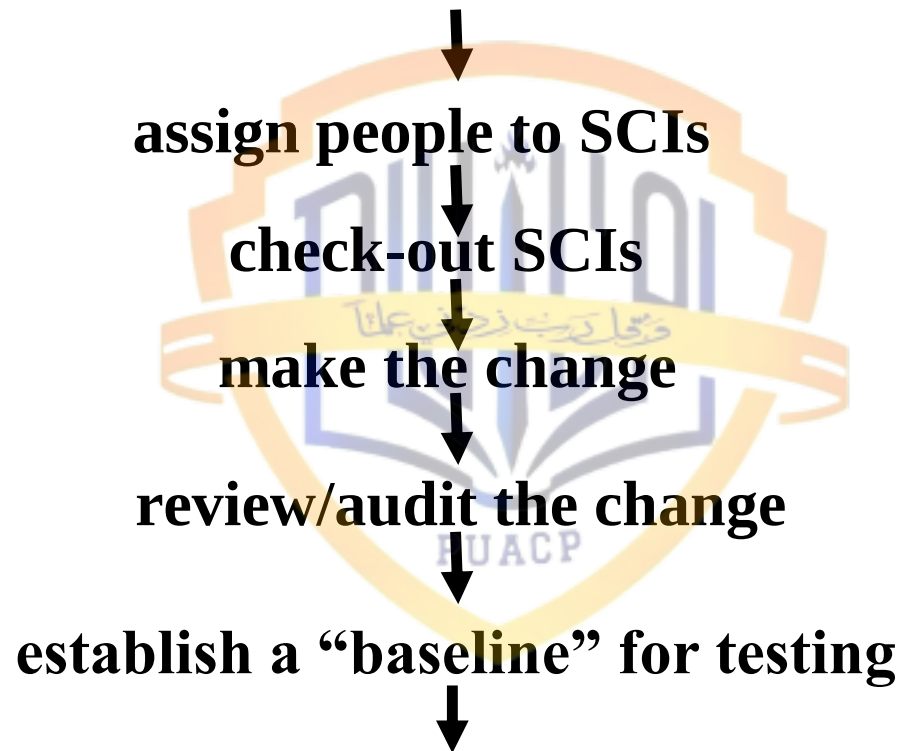
Change Control Process

The Change Control Process - 1



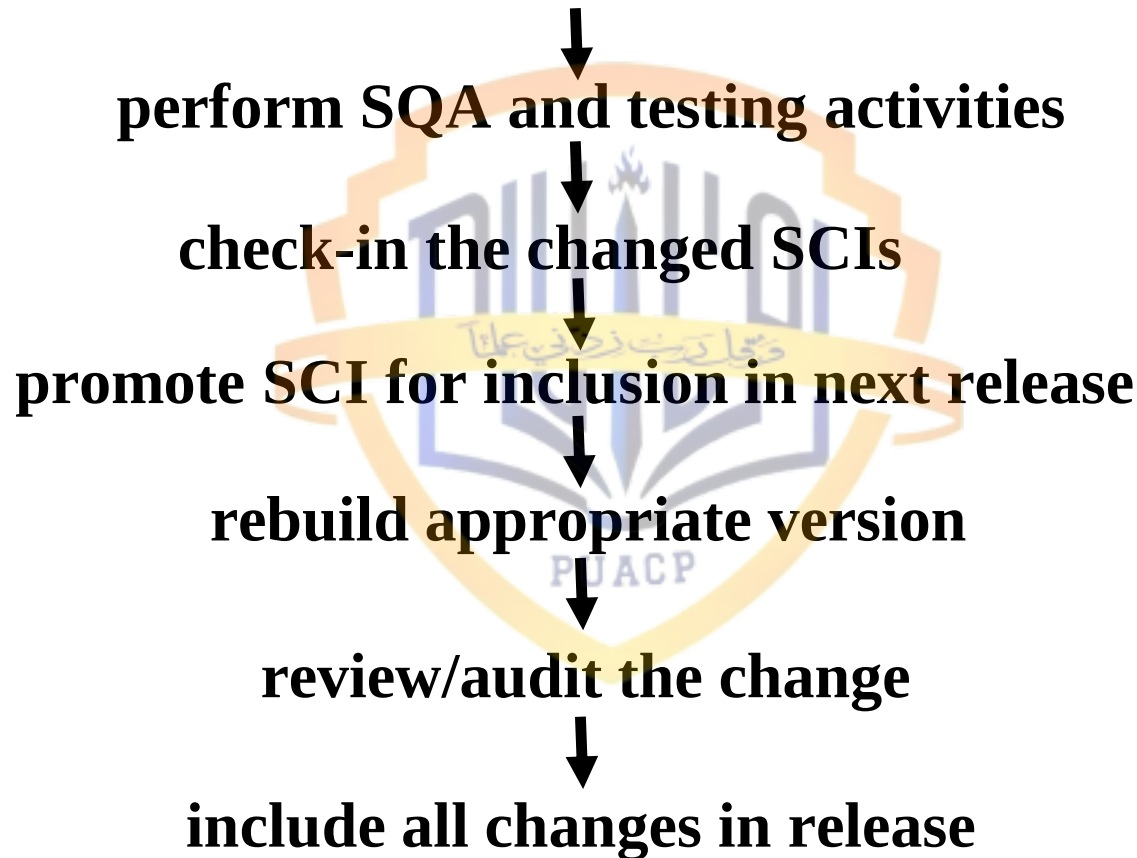
Change Control Process—2

The Change Control Process - 2

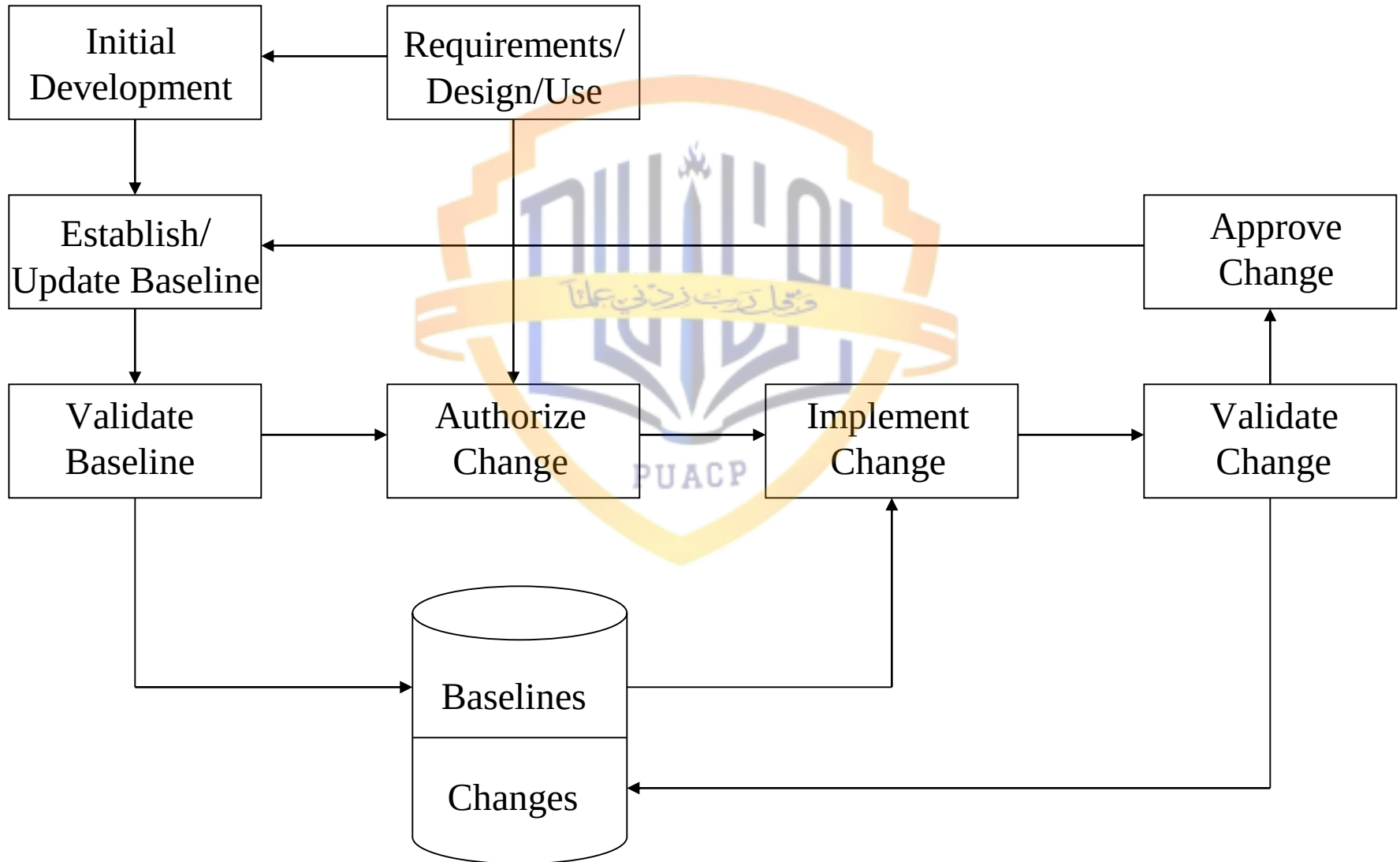


Change Control Process—3

The Change Control Process - 3



Software Configuration Management Overview



Software Configuration Management Plan



IEEE SCM Plan

- Introduction
- Reference Documents, Definitions, Acronyms
- Management
- Activities
- Resources
- Plan Maintenance
- Plan Approval



Software Quality Assurance Plan (SQAP)

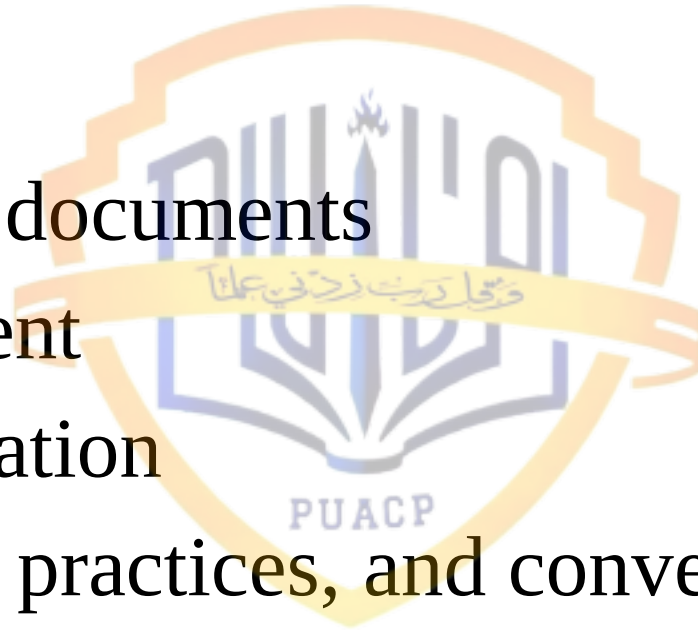


SQA Plan

- Every development and maintenance project should have a software quality assurance plan (SQAP) that specifies:
 - Its goals
 - The SQA tasks to be performed
 - The standards against which the development work is to be measured
 - The procedures and organizational structure

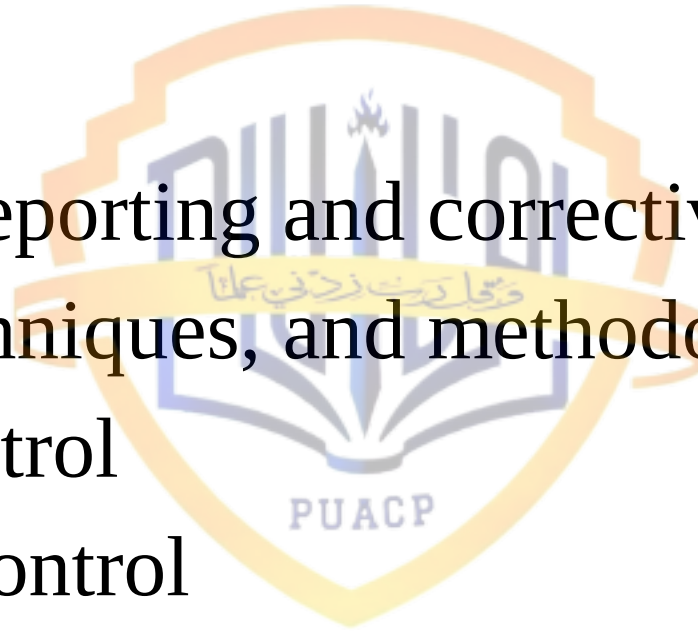
IEEE SQAP Standard - 1

- Purpose
- Reference documents
- Management
- Documentation
- Standards, practices, and conventions
- Reviews and audits
- Software configuration management



IEEE SQAP Standard - 2

- Test
- Problem reporting and corrective action
- Tools, techniques, and methodologies
- Media control
- Supplier control
- Records collection, maintenance, and retention

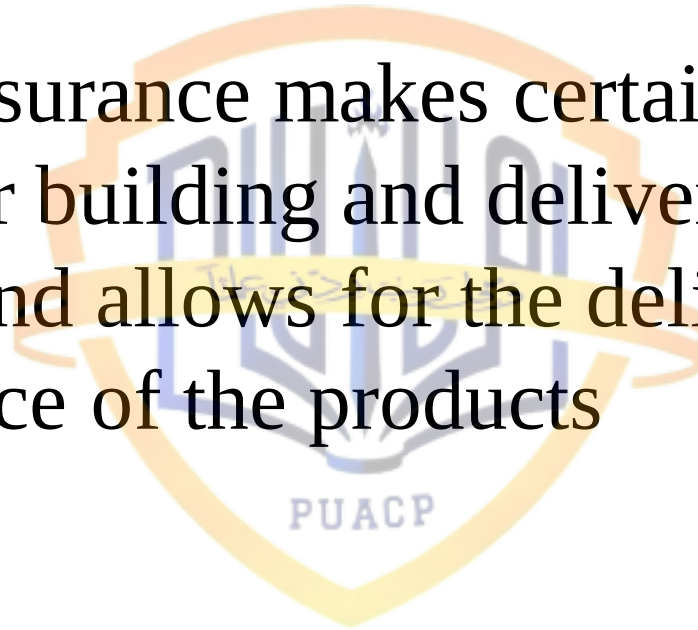


Process Assurance



Process Assurance - 0

- Process assurance makes certain that the process for building and delivering software is robust and allows for the delivery and maintenance of the products



Process Assurance - 1

- Process assurance consists of the collective activities carried out while developing a product to ensure that the methods and techniques used are integrated, consistent, and correctly applied
- Emphasis is given to cost, time, technical requirements, testing measurements, and prototyping

Process Assurance - 2

- Process assurance involves the interrelationships of several different components. Depending on how these are managed, they can have a major positive impact on the products
- Once an effective process assurance program is put in place and shown to be beneficial, then emphasis can be placed in making verification and validation strategies effective and in improving the quality of the products

Process Assurance - 3

- Successful process assurance is based on planning and organization
- There are several important aspects of planning and organization that must be considered before starting the project
- I'll show you a picture, which captures the components of planning and organization

Components of Planning and Organization



Causes of Failure in Process Assurance

- Lack of Management Support
- Lack of User Involvement
- Lack of Project Leadership
- Lack of Measures of Success

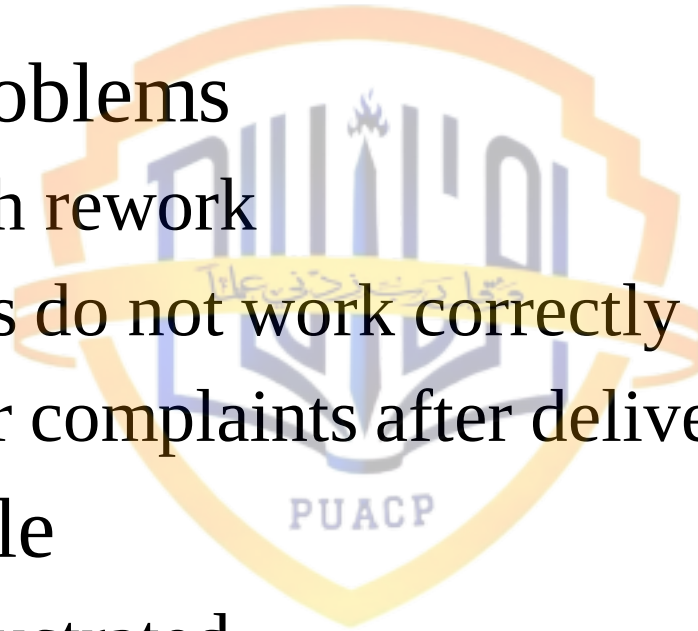


Symptoms of Process Failure - 1

- Commitments consistently missed
 - Late delivery
 - Last minute crunches
 - Spiraling costs
- No management visibility into progress
 - You're always being surprised

Symptoms of Process Failure - 2

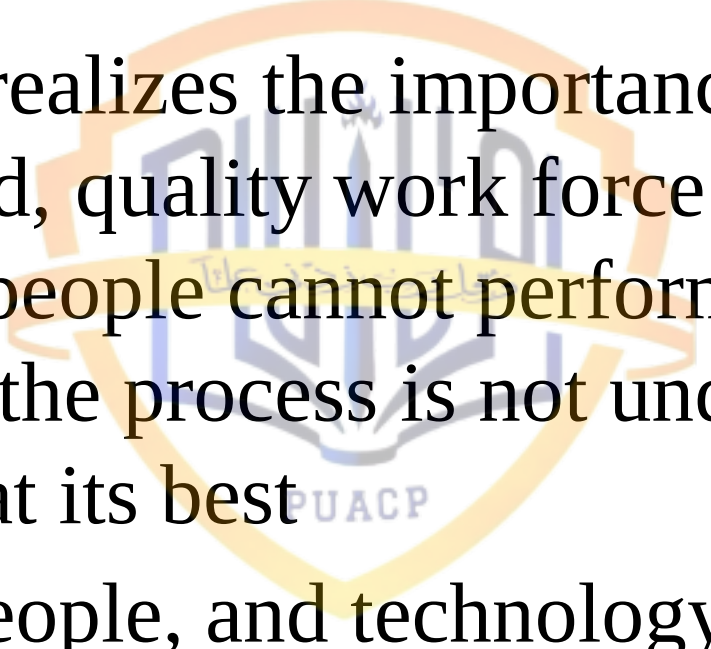
- Quality problems
 - Too much rework
 - Functions do not work correctly
 - Customer complaints after delivery
- Poor morale
 - People frustrated
 - Is anyone in charge?

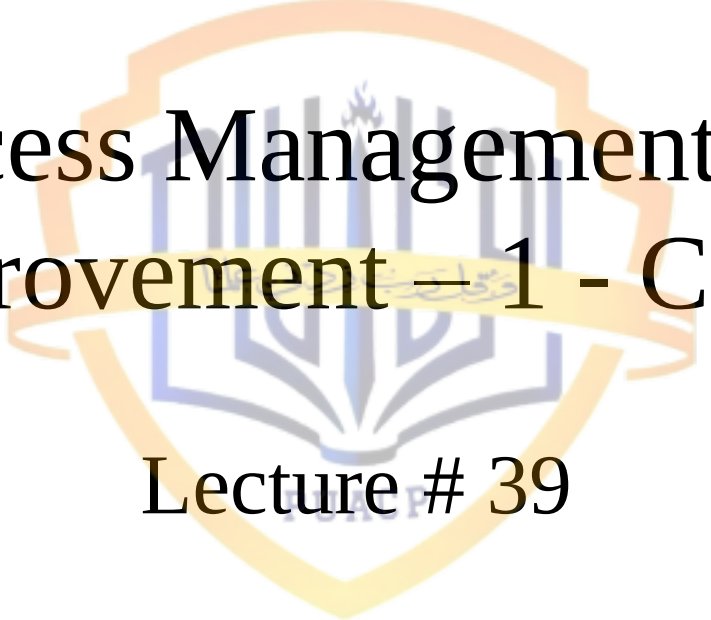


Common Misconceptions

- I don't need process, I have
 - Really good people
 - Advanced technology
 - An experience manager
- Process
 - Interfaces with creativity
 - Equals bureaucracy + regimentation
 - Isn't needed when building prototypes
 - Is only useful on large projects
 - Hinders agility in fast-moving markets
 - Costs too much



- 
- A large, semi-transparent watermark of the PUACP logo is centered in the background. It features a shield with Arabic calligraphy and the acronym 'PUACP' at the bottom.
- Everyone realizes the importance of having a motivated, quality work force but even our finest people cannot perform at their best when the process is not understood or operating at its best
 - Process, people, and technology are the major determinants of product cost, schedule, and cost

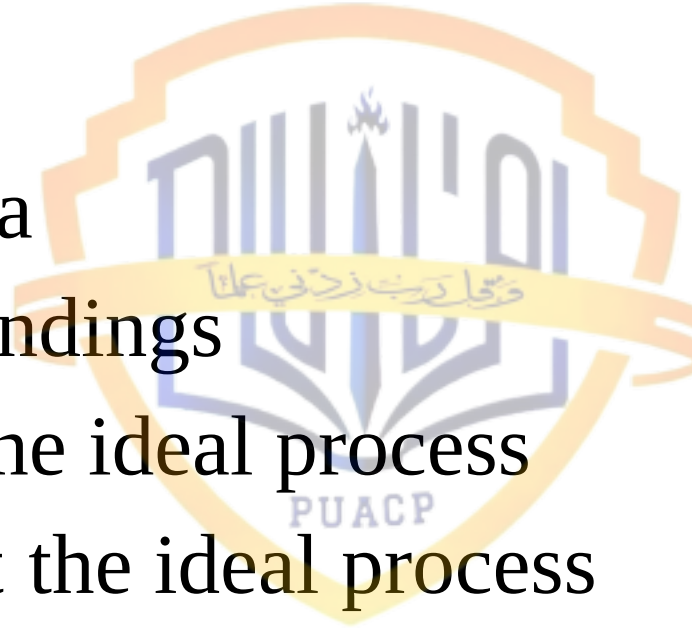
The logo of the University of Al-Qadisiyah is a shield-shaped emblem. It features a stylized open book at the base, with a flame or torch rising from the center. The shield is outlined in orange and yellow, and contains Arabic calligraphy in blue and gold.

Process Management and Improvement – 1 - CMM

Lecture # 39

Seven Steps of the Process Improvement

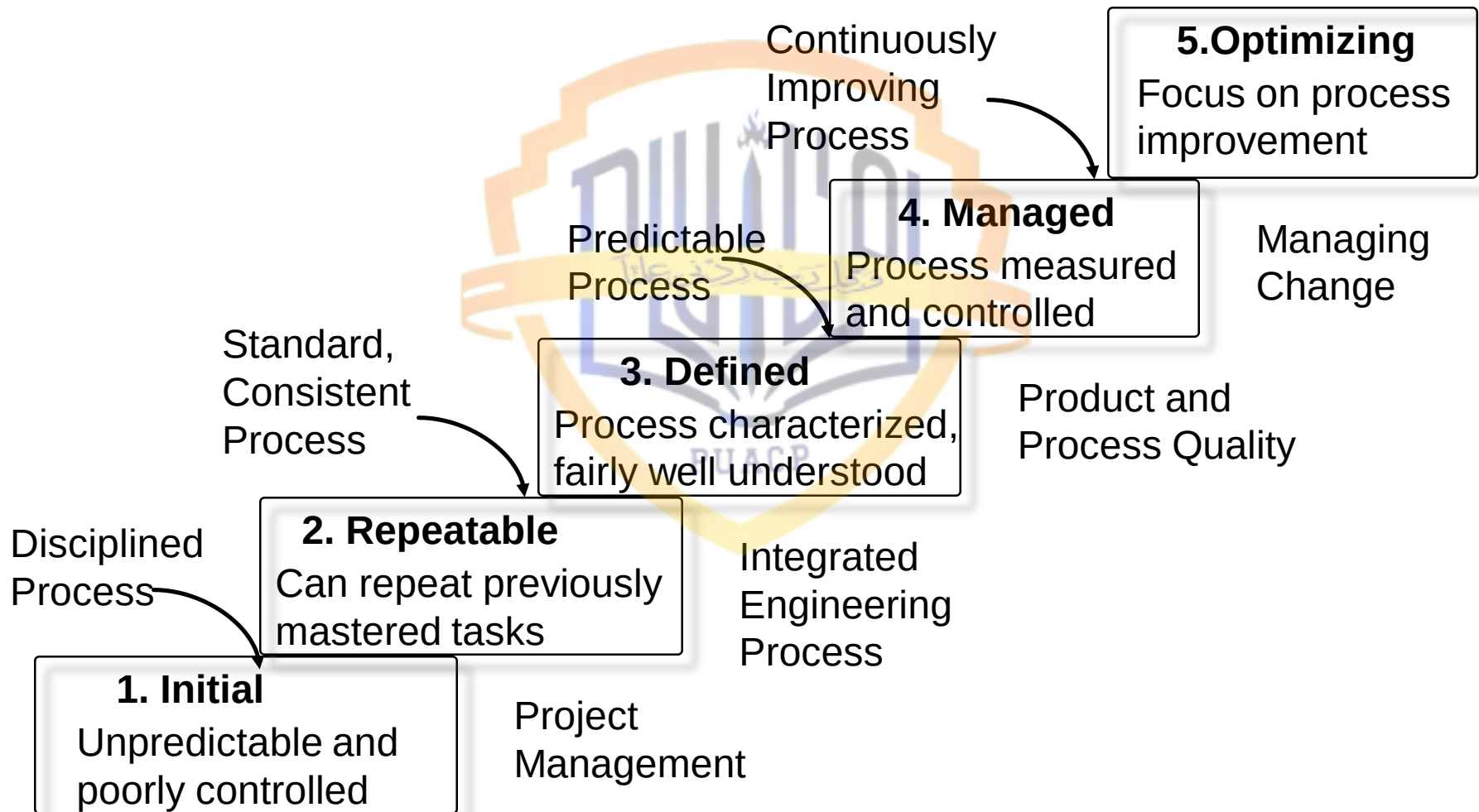
- Plan
- Gather data
- Analyze findings
- Describe the ideal process
- Implement the ideal process
- Measure progress
- Standardize the process



Capability Maturity Model



The Five Levels of Software Process Maturity



Levels/ Process Categories	Management	Organizational	Engineering
5 Optimizing		Technology Change Management Process Change Management	Defect Prevention
4 Managed	Quantitative Software Management		Software Quality Management
3 Defined	Integrated Software Management Intergroup Coordination	Organization Process Focus Organization Process Definition Training Program	Software Product Engineering Peer Reviews
2 Repeatable	Requirements Management Software Project Planning Software Project Tracking and Oversight Software Subcontract Management Software Quality Assurance Software Configuration Management		
1 Initial		Ad Hoc Processes	

Capability Maturity Model Integration



Staged Representation

- The staged representation is the approach used in the Software CMM. It is an approach that uses predefined sets of process areas to define an improvement path for an organization. This improvement path is described by a model component called a maturity level
- A maturity level is a well-defined evolutionary plateau toward achieving improved organizational processes

Continuous Representation

- The continuous representation is the approach used in the SECM and the IPD-CMM. This approach allows an organization to select a specific process area and improve relative to it
- The continuous representation uses capability levels to characterize improvement relative to an individual process area

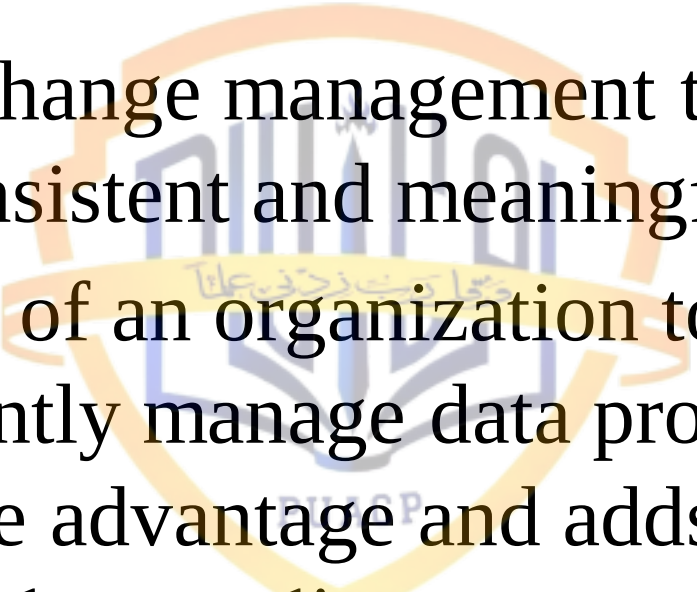
CMMI Model Structure

- Maturity Levels (staged representation) or Capability Levels (continuous representation)
- Process Areas
- Goals – Generic and Specific
- Practices – Generic and Specific

Introduction to Quality Metrics - 1

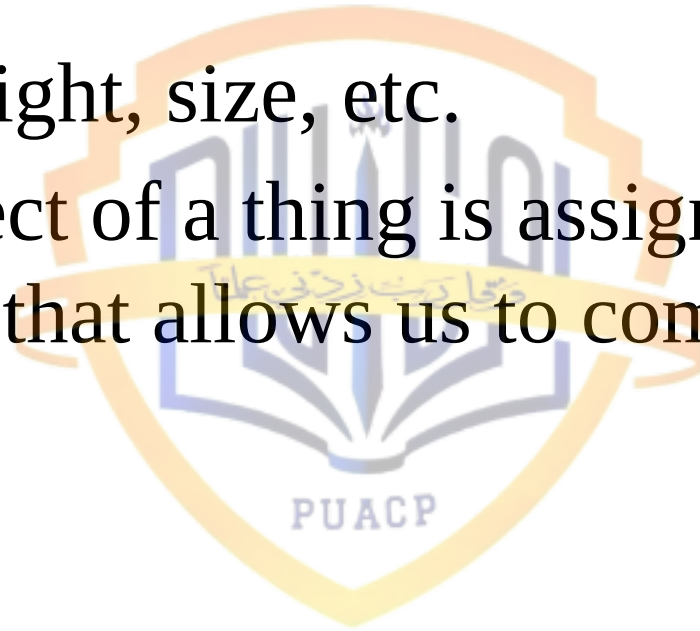
Lecture # 41-43

- The software industry is in a constant state of change. With advances in hardware, software, graphical, and multimedia technologies, applications are constantly being rethought, redesigned, and reengineered. New development methods and techniques are continuously evolving. Because of this environment of continuous change, the software environment or software business can be extremely difficult to manage

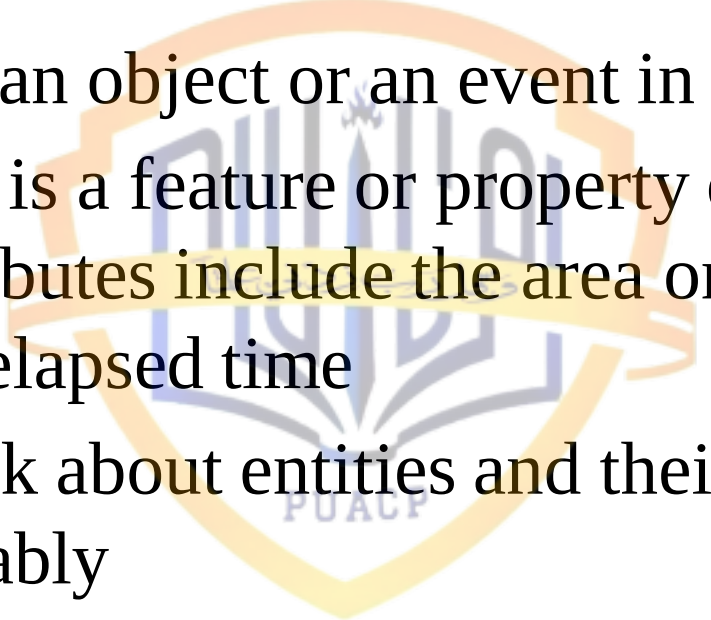
- 
- Effective change management techniques require consistent and meaningful measures
 - The ability of an organization to effectively and efficiently manage data provides a true competitive advantage and adds value to the company's bottom line

- 
- The logo of the Punjab University of Agriculture, Chaudhary Sahib Singh Campus, Ludhiana (PUACP) is centered in the background. It features a shield-like emblem with a yellow border. Inside the shield, there is a blue and white design that includes an open book and a sun-like symbol. The acronym 'PUACP' is written in blue at the bottom of the emblem. Above the emblem, there is a banner with Urdu text. The list of bullet points is overlaid on the left side of the slide.
- Measurement lies at the heart of many systems that govern our lives
 - Economic measurements
 - Military measurements
 - Medical measurements
 - Atmospheric measurements
 - Measurements in every day life

- Prices, weight, size, etc.
- Some aspect of a thing is assigned a descriptor that allows us to compare it with others

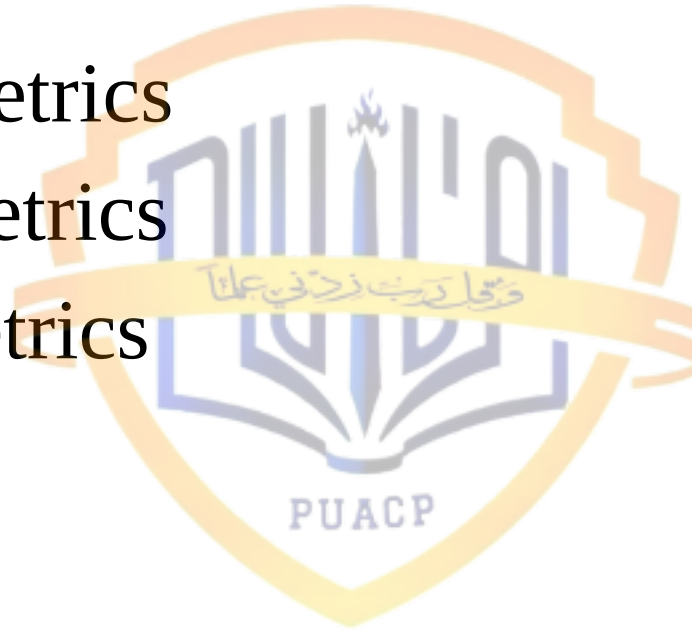


- Measurement is the process by which numbers or symbols are assigned to attributes of entities in the real world in such a way as to describe them according to clearly defined rules

- 
- An entity is an object or an event in the real world
 - An attribute is a feature or property of an entity. Typical attributes include the area or color, the cost, or the elapsed time
 - We often talk about entities and their attributes interchangeably
 - ‘It is cold today’
 - ‘Usman is taller than Ali’

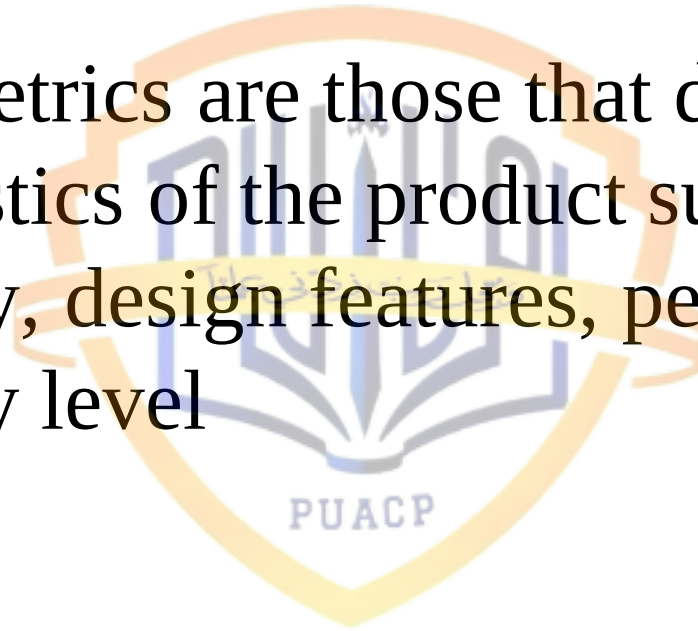
Categories of Software Metrics

- Product metrics
- Process metrics
- Project metrics



Product Metrics

- Product metrics are those that describe the characteristics of the product such as size, complexity, design features, performance, and quality level



Process Metrics

- Process metrics are those that can be used for improving the software development and maintenance process
- Examples include the effectiveness of defect removal during development, the pattern of testing defect arrival, and the response time of the fix process

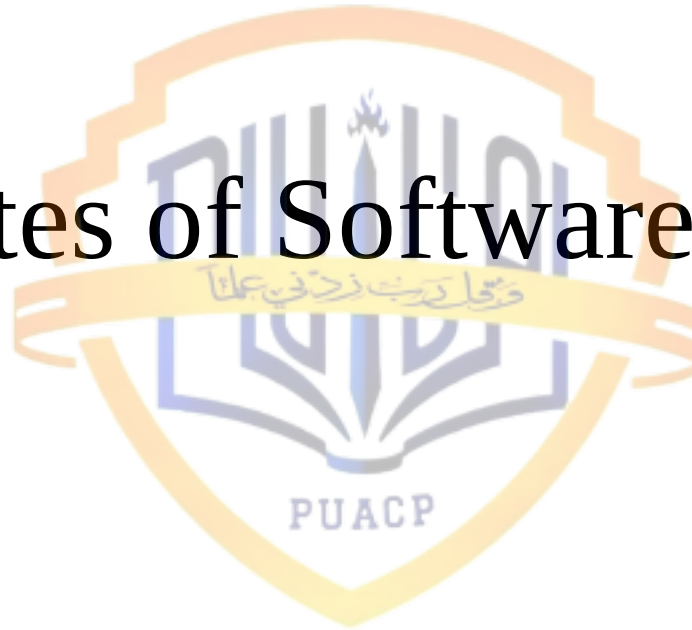
Project Metrics

- Project metrics are those that describe the project characteristics and execution
- Examples include the number of software developers, the staffing pattern over the life cycle of the software, cost, schedule, and productivity

Software Quality Metrics

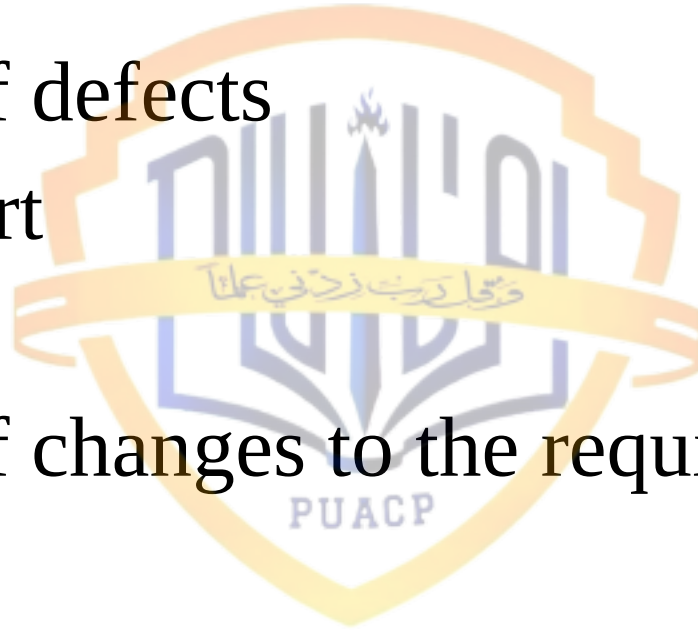
- Software quality metrics are a subset of software metrics that focus on quality aspects of the product, process, and project
- In general, software metrics are more closely associated with process and product metrics than with project metrics
- Nonetheless, the project parameters such as number of developers and their skill levels, the schedule, the size, and the organization structure certainly affect the quality of the product

Attributes of Software Quality



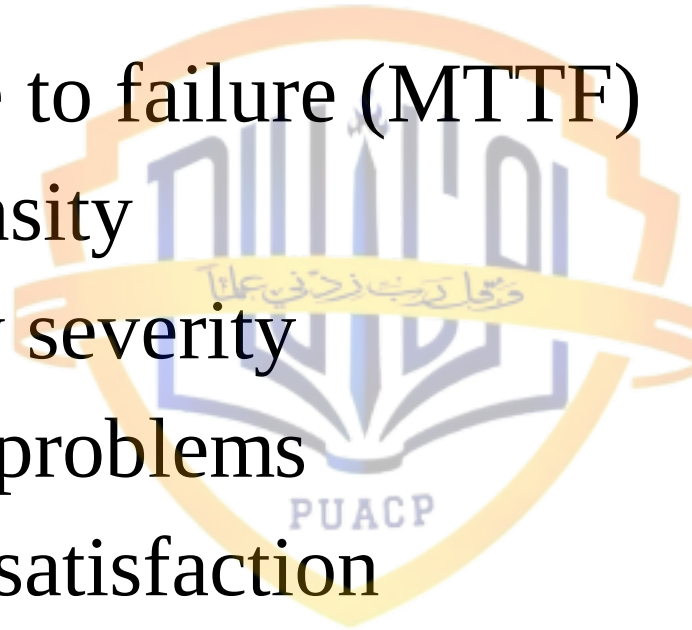
Seven Commonly Tracked Measures

- Number of defects
- Work effort
- Schedule
- Number of changes to the requirements
- Size
- Documentation defects
- Complexity

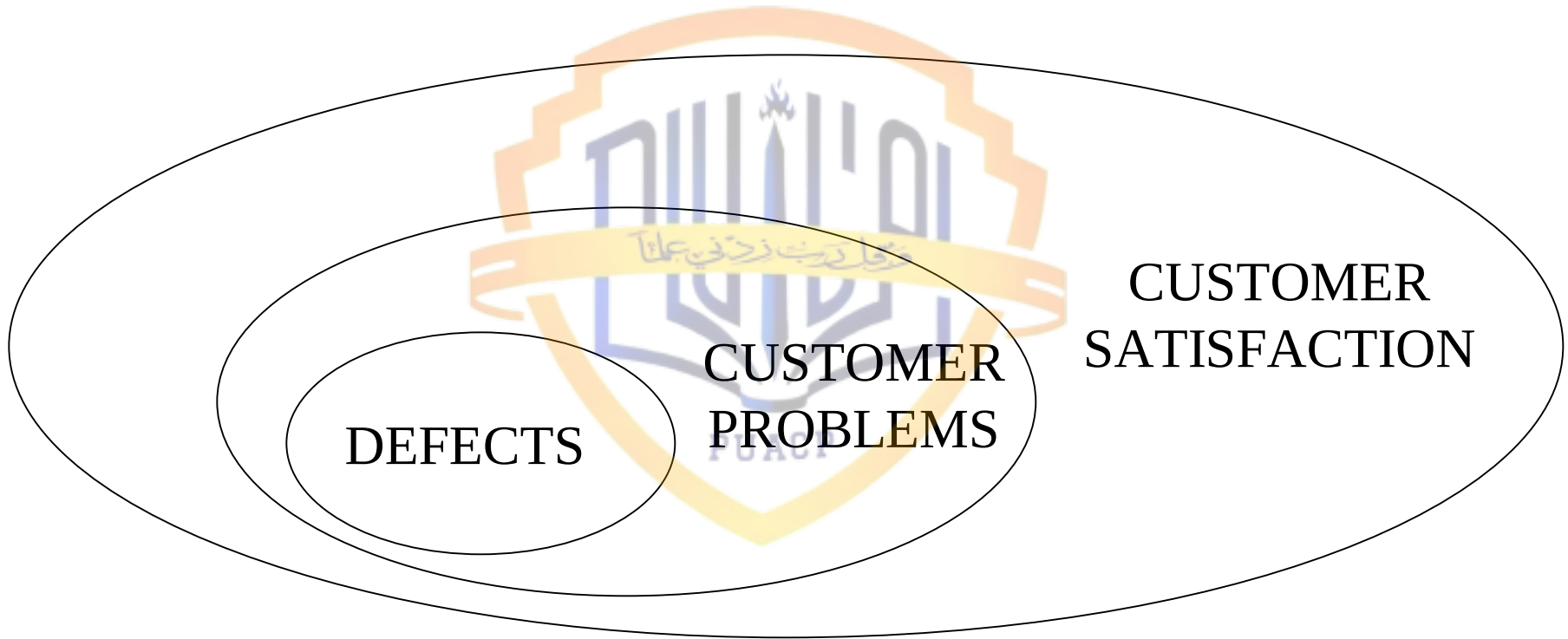


Additional Software Quality Metrics

- Mean time to failure (MTTF)
- Defect density
- Defects by severity
- Customer problems
- Customer satisfaction



Scopes of Three Quality Metrics

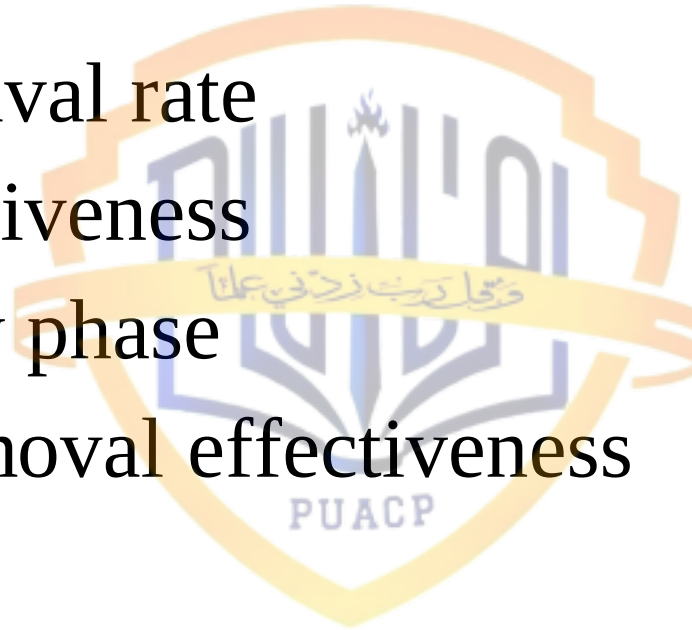


Process Quality Metrics

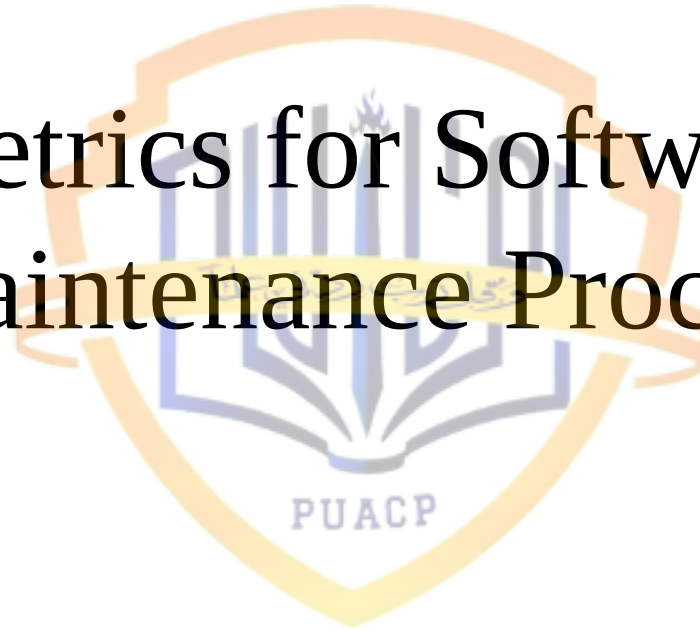


Process Metrics

- Defect arrival rate
- Test effectiveness
- Defects by phase
- Defect removal effectiveness

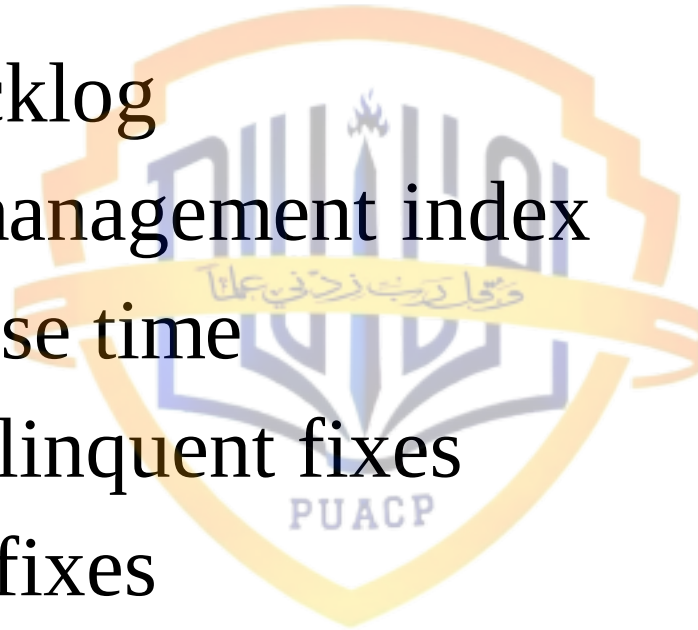


Metrics for Software Maintenance Process



Metrics in Software Maintenance

- Defect backlog
- Backlog management index
- Fix response time
- Percent delinquent fixes
- Defective fixes



Function Points



Function Points - 5

- A function point can be defined as a collection of executable statements that performs a certain task, together with declarations of the formal parameters and local variables manipulated by those statements
- The function points were proposed by Albrecht and his colleagues at IBM in mid-1970s

Function Points - 6

- It addresses some of the problems associated with LOC counts in size and productivity measures, especially the differences in LOC counts due to different levels of languages used
- It is a weighted total of five major components that comprise an application

Function Points - 7

- These five components are
 - Number of external inputs
 - Number of external outputs
 - Number of logical internal files
 - Number of external interface files
 - Number of external inquiries

External Inputs

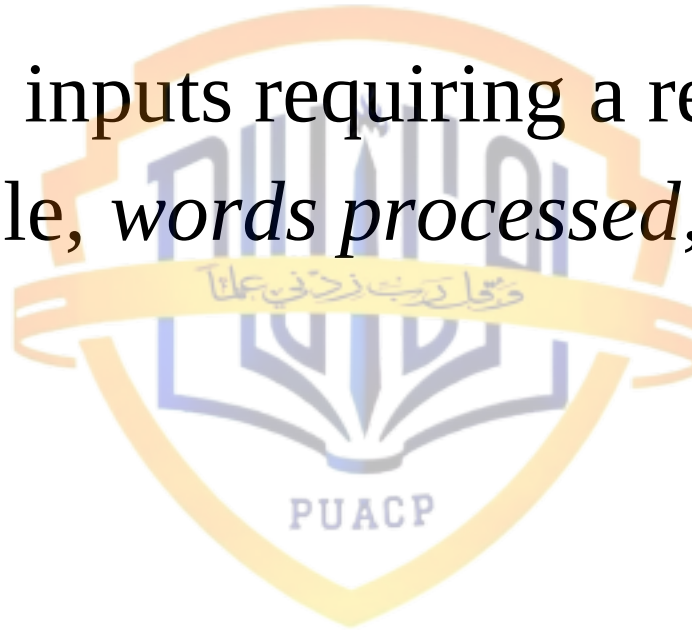
- Those items provided by the user that describe distinct application-oriented data (such as file names and menu selections). These items do not include inquiries, which are counted separately
- For example, *document filename*, *personal dictionary-name*

External Outputs

- Those items provided to the user that generate distinct application-oriented data (such as reports and messages, rather than the individual components of these)
- For example, *misspelled word report*, *number-of-words-processed message*, *number-of-errors-so-far message*

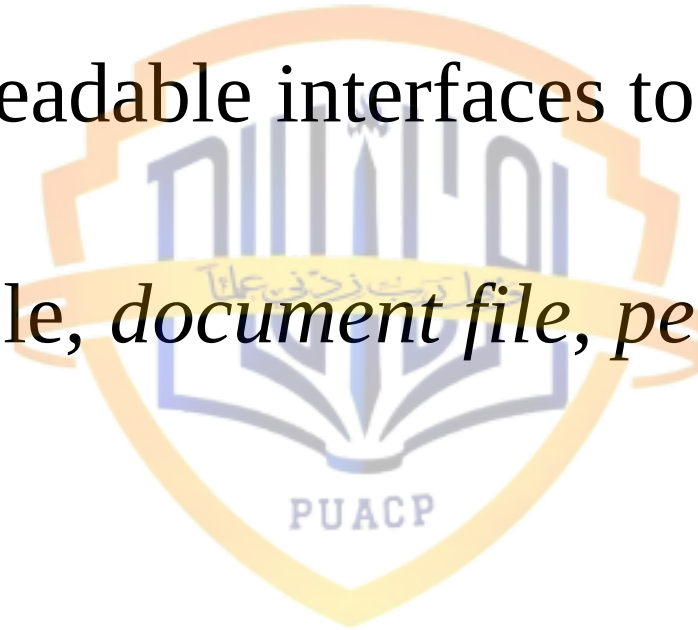
External Inquiries

- Interactive inputs requiring a response
- For example, *words processed, errors so far*



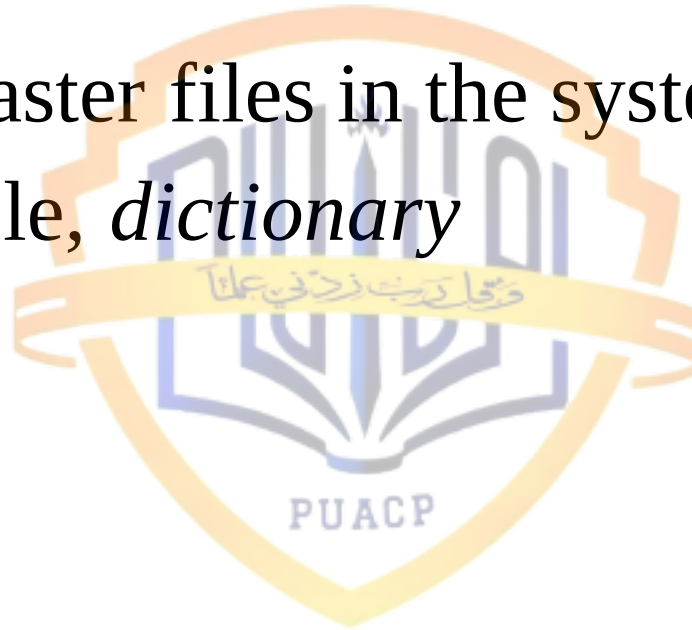
External Files

- Machine-readable interfaces to other systems
- For example, *document file*, *personal dictionary*



Internal Files

- Logical master files in the system
- For example, *dictionary*



Function Points - 8

- Each item/component is assigned a subjective “complexity” rating on a three point ordinal scale: “simple”, “average”, or “complex”
- Then a weight is assigned to the item according to the following table

The logo of the University of Al-Qadisiyah is a shield-shaped emblem. It features a stylized open book at the base, with a flame or torch rising from its center. Above the book, there is a banner with Arabic calligraphy. The entire emblem is enclosed within a shield with a serrated top edge. The colors are primarily blue, yellow, and orange.

Following slide to be inserted

FP Complexity Weights

FP Complexity Weights

Item	Simple	Weighting Factor Average	Complex
External Inputs	3	4	6
External Outputs	4	5	7
External inquiries	3	4	6
External files	7	10	15
Internal files	5	7	10

Function Points - 9

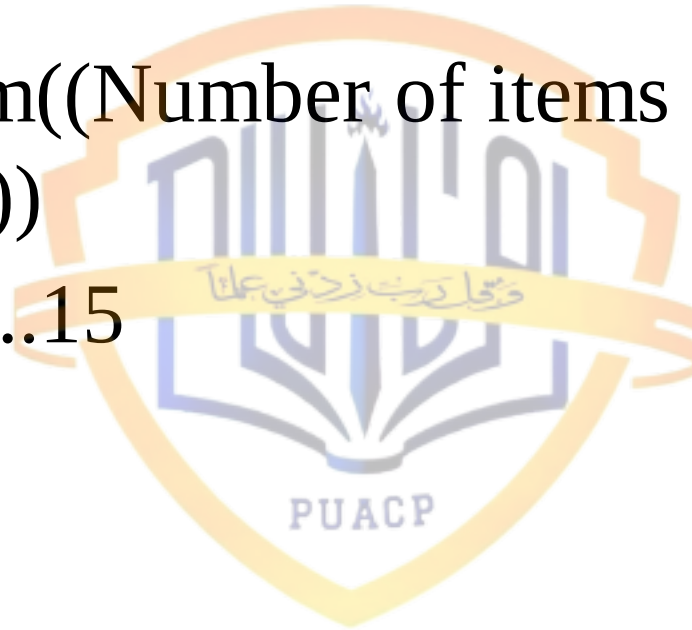
- The complexity classification of each component is based on a set of standards that define complexity in terms of objective guidelines
- For instance, for the external output component, if the number of data element types is 20 or more and the number of file types referenced is two or more, then complexity is high
- If the number of data element types is 5 or fewer and the number of file types referenced is two to three

Function Points - 10

- With the weighting factors, the first step is to calculate the Unadjusted function counts (UFC) based on the formula, which is obtained by multiplying the weighting factors with the corresponding components, and adding them up
- There are fifteen (15) different varieties of items/components – in theory

Unadjusted Function Counts

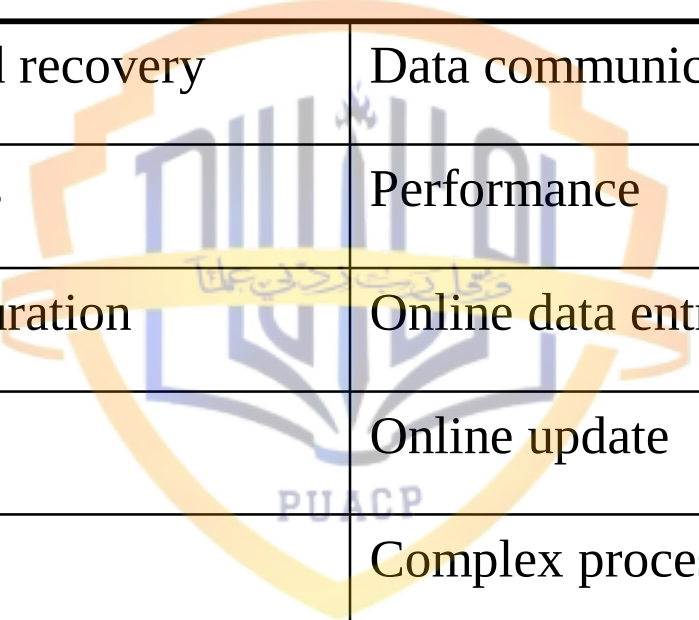
- $UFC = \sum((\text{Number of items of variety } i) * \text{weight}(i))$
 - For $i = 1..15$



Function Points - 11

- We calculate an adjusted function point count, FP, by multiplying UFC by a technical complexity factor, TCF
- This technical involves the 14 contributing factors

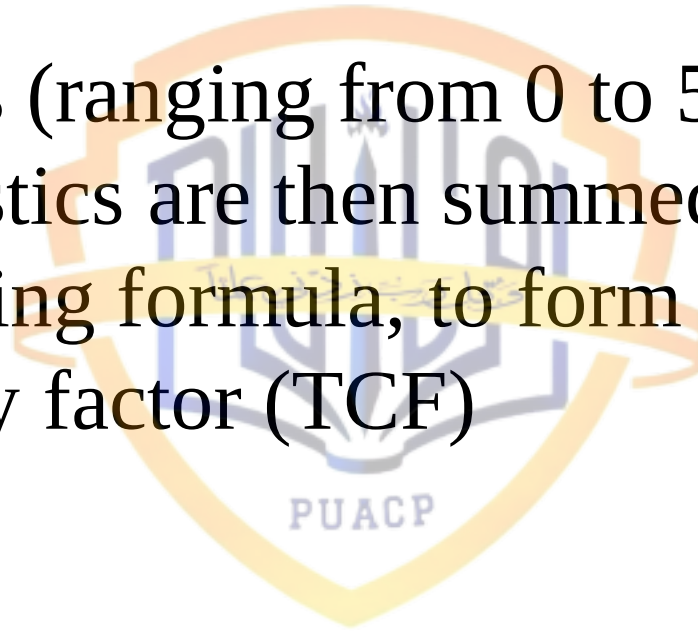
Components of the Technical Complexity Factor



Reliable back-up and recovery	Data communications
Distributed functions	Performance
Heavily used configuration	Online data entry
Operational ease	Online update
Complex interface	Complex processing
Reusability	Installation ease
Multiple sites	Facilitate change

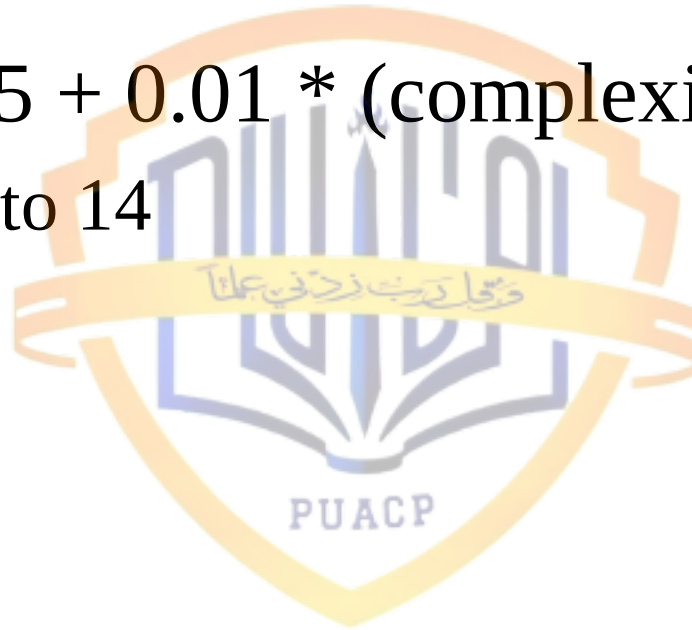
Function Points - 12

- The scores (ranging from 0 to 5) for these characteristics are then summed, based on the following formula, to form the technical complexity factor (TCF)



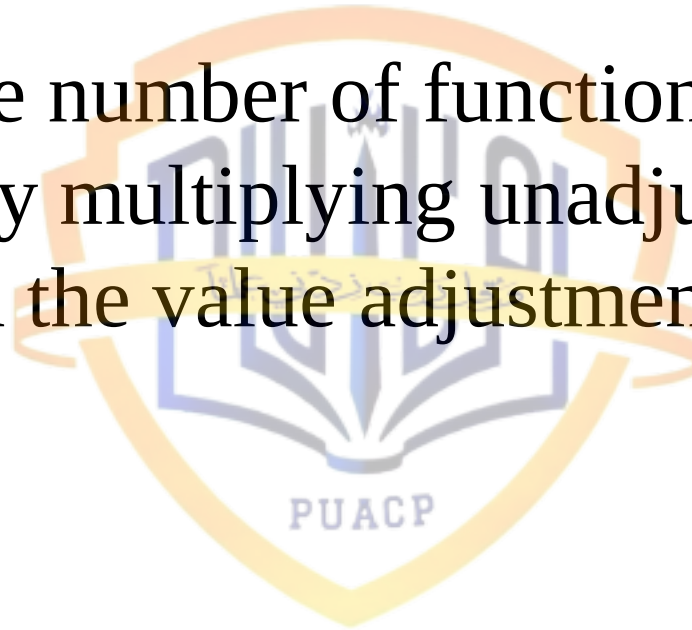
Technical Complexity Factor

- $TCF = 0.65 + 0.01 * (\text{complexity of item } i)$
 - For $i = 1$ to 14

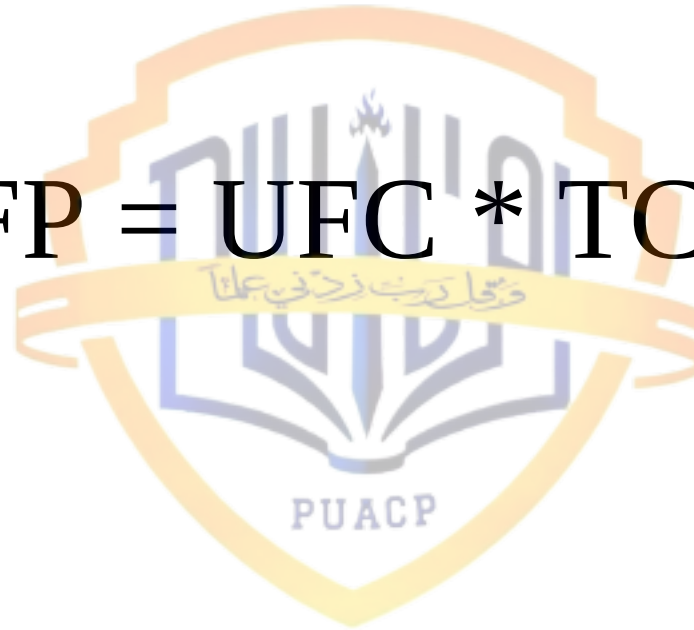


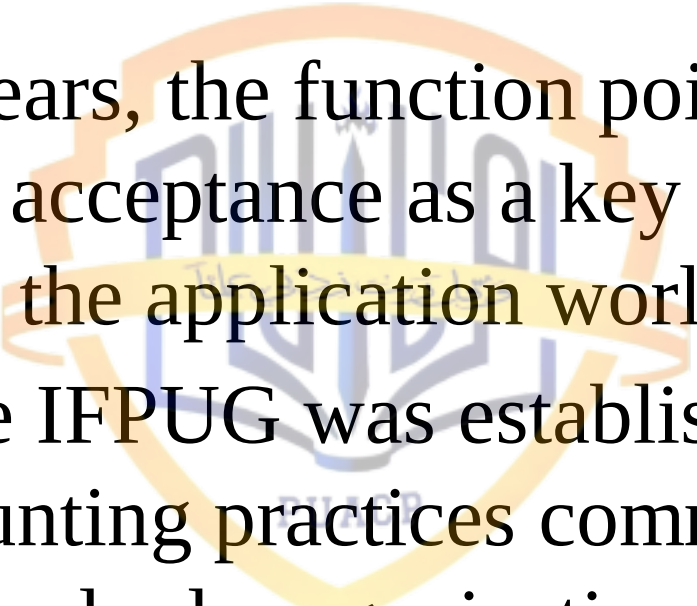
Function Points - 13

- Finally, the number of function points is obtained by multiplying unadjusted function counts and the value adjustment factor:



$$FP = UFC * TCF$$



- 
- Over the years, the function point metric has gained acceptance as a key productivity measure in the application world
 - In 1986 the IFPUG was established. The IFPUG counting practices committee is the *de facto* standards organization for function point counting methods

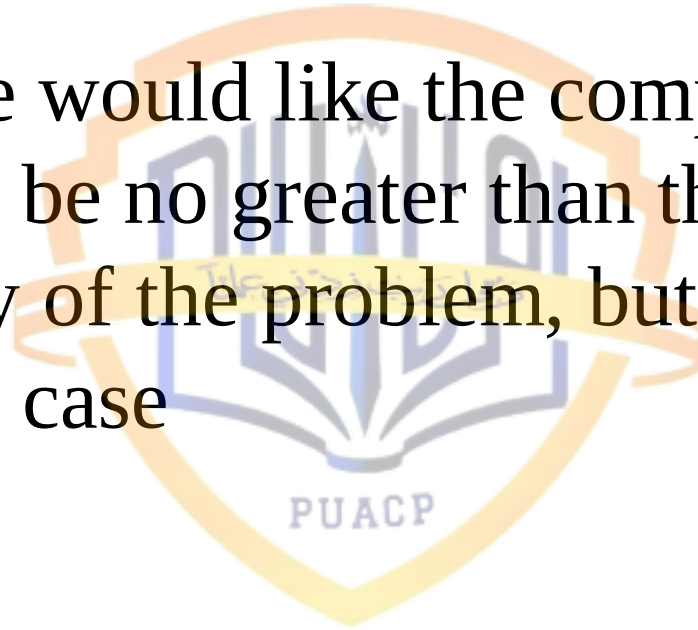
The logo of the University of Al-Qadisiyah is a shield-shaped emblem. It features a stylized open book at the base, with a flame or torch rising from its center. Above the book, there is a banner with Arabic calligraphy. The entire emblem is enclosed within a shield with a serrated top edge. The colors are primarily blue, yellow, and orange.

Complexity

Say few words on complexity

Complexity - 1

- Ideally, we would like the complexity of the solution to be no greater than the complexity of the problem, but that is not always the case

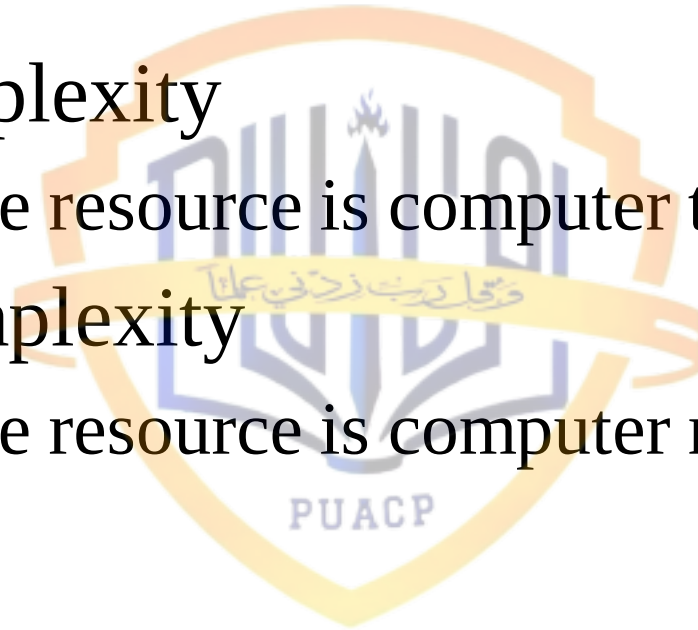


Complexity - 2

- The complexity of a problem as the amount of resources required for an optimal solution to the problem
- The complexity of a solution can be regarded in terms of the resources needed to implement a particular solution

Complexity - 3

- Time complexity
 - Where the resource is computer time
- Space complexity
 - Where the resource is computer memory



Complexity - 4

- Problem complexity
- Algorithmic complexity
- Structural complexity
- Cognitive complexity



Cyclomatic Complexity



Cyclomatic Complexity - 1

- The measurement of cyclomatic complexity by McCabe was designed to indicate a program's testability and understandability (maintainability)
- It is a classical graph theory cyclomatic number, indicating the number of regions in a graph

Cyclomatic Complexity

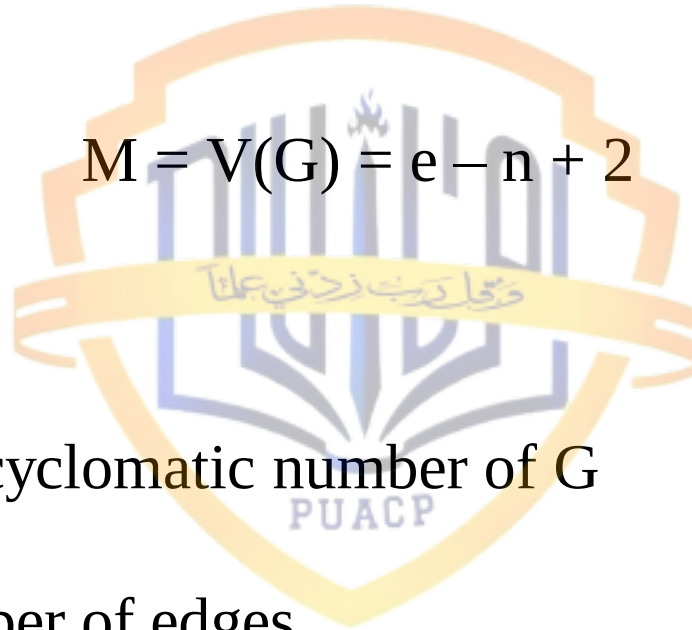
$$M = V(G) = e - n + 2$$

where

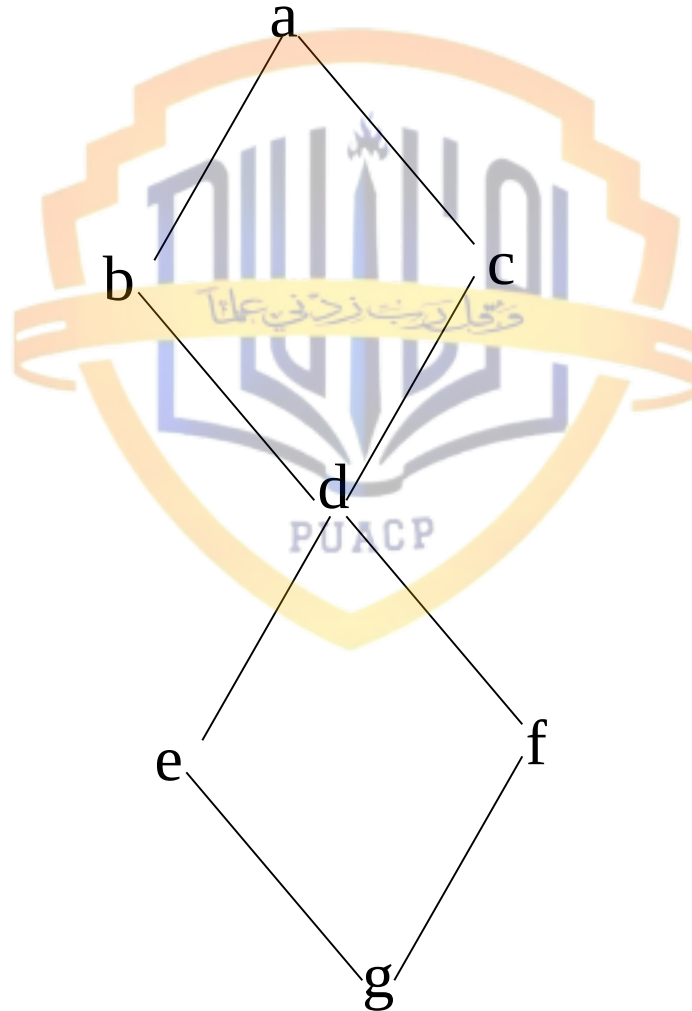
$V(G)$ = cyclomatic number of G

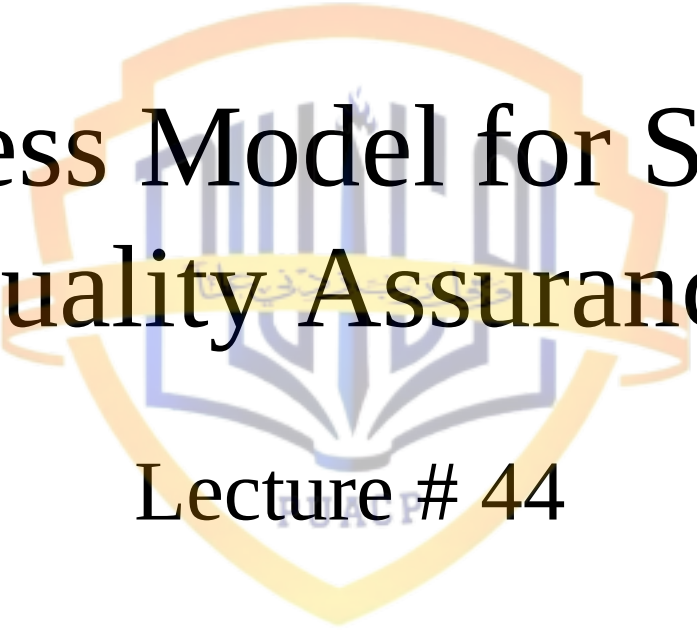
e = number of edges

n = number of nodes



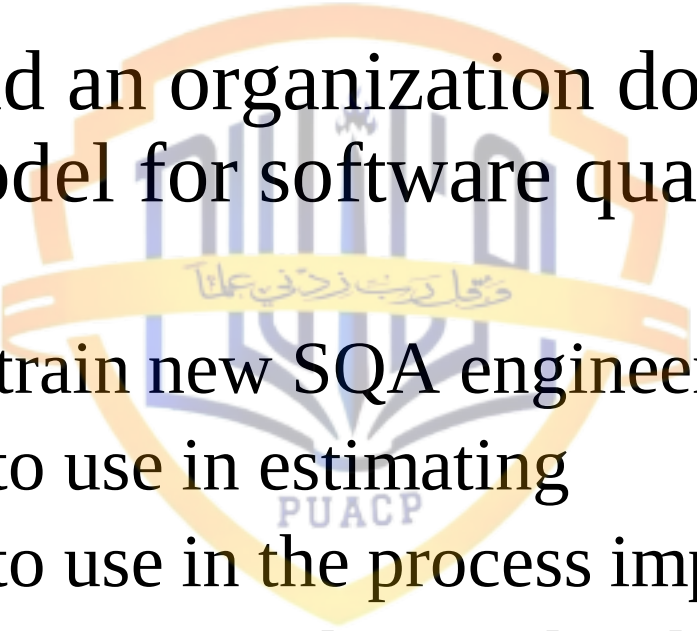
Simple Control Graph Example



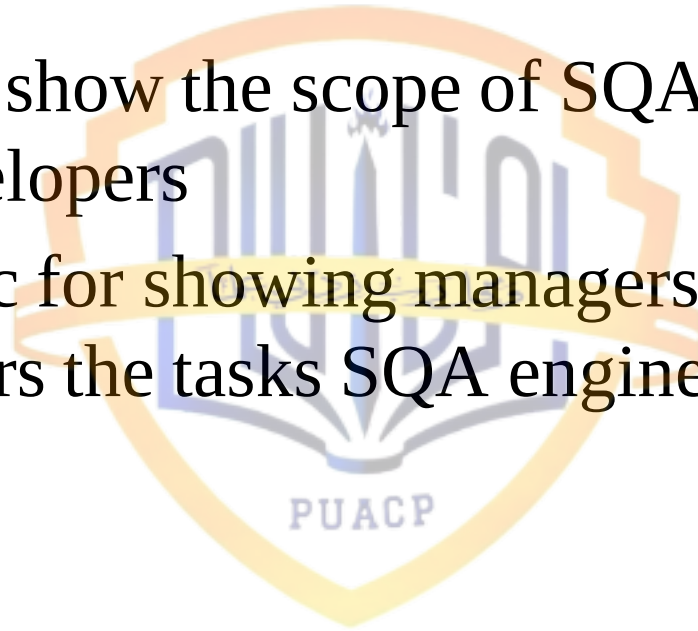
The logo of the University of Jordan is a watermark in the background. It features a shield with an orange border. Inside the shield, there is a blue emblem with Arabic calligraphy and a yellow banner across the middle. The text on the banner is in Arabic.

A Process Model for Software Quality Assurance

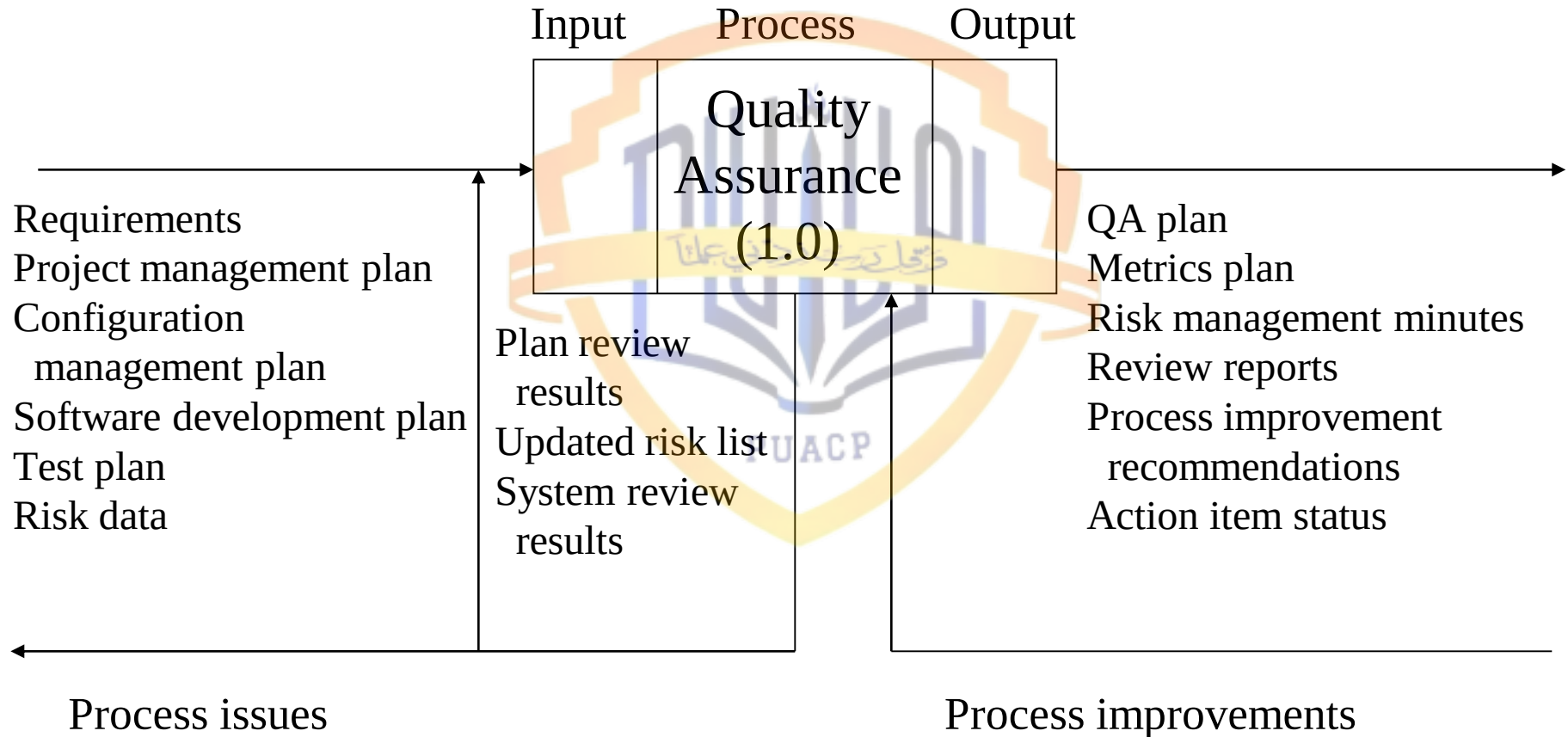
Lecture # 44

- 
- What would an organization do with a process model for software quality assurance?
 - A tool to train new SQA engineers
 - A model to use in estimating
 - A model to use in the process improvement
 - A model to use to select or develop a set of standards and templates for SQA documentation

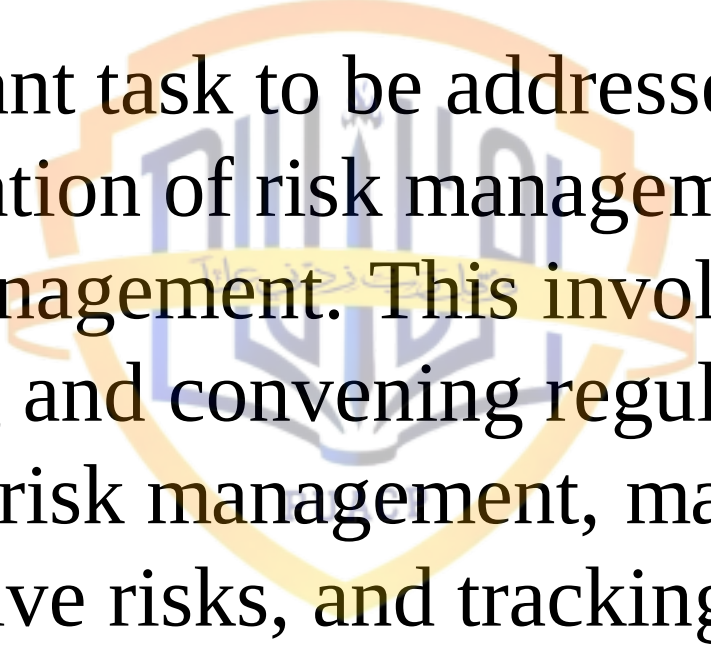
- A tool to show the scope of SQA activities to new developers
- A graphic for showing managers and developers the tasks SQA engineers perform

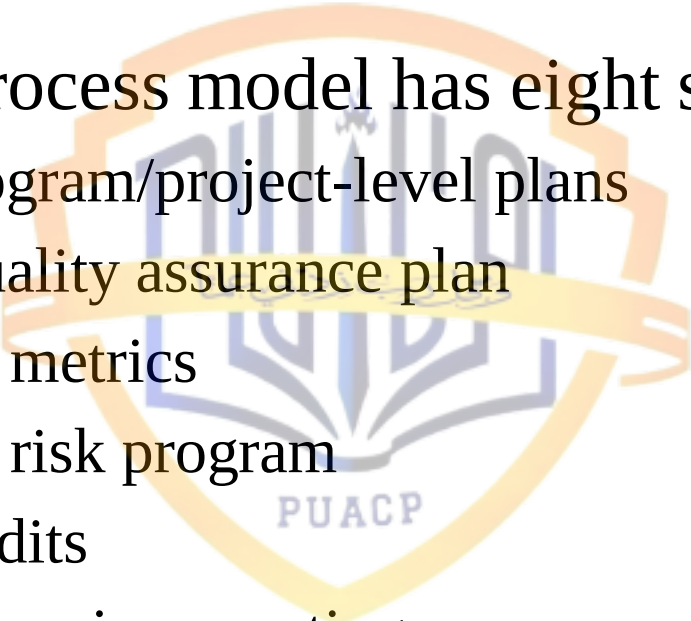


Software Quality Assurance Level 0 IPO Diagram



- The SQA process is based on relationships, interfaces, and tasks that are defined in other project plans (for example, project management plan, configuration management plan)

- 
- A significant task to be addressed by SQA is coordination of risk management to assist project management. This involves scheduling and convening regular meetings to address risk management, maintaining lists of active risks, and tracking status of risk mitigation plans for project management

- 
- A large, semi-transparent watermark of the PUACP logo is centered in the background. It features a shield-like shape with a book icon and the text 'PUACP' below it.
- This SQA process model has eight sub- processes
 - Review program/project-level plans
 - Develop quality assurance plan
 - Coordinate metrics
 - Coordinate risk program
 - Perform audits
 - Coordinate review meetings
 - Facilitate process improvement
 - Monitor test program

Software Quality Assurance Level 1 IPO Diagram

